scientific reports

Check for updates

OPEN

Multi-strategy cooperative enhancement dung beetle optimizer and its application in obstacle avoidance navigation

Xiaojie Tang✉, Zhengyang He & Chengfen Jia

Path planning is a crucial component of unmanned driving systems designed to address the challenge of autonomous navigation. This article introduces a novel approach called the multi-strategy cooperative enhanced dung beetle optimization algorithm (RCDBO) and applies it to dispose of path planning issues. Its primary aim is to mitigate the shortcomings of the fundamental dung beetle optimization algorithm (DBO), namely, its tendency to prematurely converge to local optima and its limited global planning capability. Initially, population initialization leverages Bernoulli-based chaotic mapping to enhance diversity and ensure uniform randomness, thereby improving both the quality of initial solutions and optimization efficiency. Subsequently, a random walk strategy is employed to perturb the rolling behavior of the dung beetle population during the initial stage, thus mitigating potential algorithmic local stagnation. Finally, adopting a vertical and horizontal crossover strategy introduces perturbations to the current best value of the dung beetle population, thereby enhancing the DBO method's global optimization capability during the later stages of evolution. The RCDBO method was evaluated against several well-established swarm intelligence algorithms using twelve benchmark test functions, the CEC2021 test suite, the Wilcoxon rank-sum test, and the Friedman test, to rigorously assess its feasibility. Additionally, the effectiveness of each strategy within the RCDBO framework was validated, and the algorithm's capability to achieve an optimal balance between global exploration and local exploitation was systematically analyzed through the CEC2021 test suite. Furthermore, in path planning simulation experiments targeting maps of varying sizes and complexities, the RCDBO algorithm, compared to the basic DBO algorithm, results in shorter planned path lengths. In simpler map environments, it exhibits fewer turning points and smoother paths, indicating that the RCDBO algorithm also possesses certain advantages in addressing path-planning challenges.

The advancement of unmanned technology is an unavoidable trend for the future intelligent transport system. The path planning problem is the technology needed to achieve autonomous navigation in unmanned driving, which is the ability of an unmanned vehicle to efficiently plan a route from one location to another and safely avoid all obstacles based on a model of the surrounding environment[1]. Owing to factors like the copiousness of environmental data and the abundance of obstacles, the path planning conundrum typically persists, established as an NP-Hard (non-deterministic polynomial hard) problem[2].

Traditional algorithms, including the A-star algorithm, Dijkstra algorithm, RRT algorithm, artificial potential field method, neural network algorithm, and others, are commonly employed in tackling the path planning challenge[3,4]. Furthermore, numerous scholars have dedicated efforts to refining these traditional algorithms. For instance, DA-star, a path-planning algorithm developed by He Z et al.[5], combines the A-star algorithm with ship navigation rules to generate optimal paths for avoiding dynamic and static obstacles in complex navigation scenarios. This method strikes a balance between minimizing navigational risks and maximizing economic benefits. Sundarraj S et al.[6] suggested the weight-controlled particle swarm-optimized Dijkstra algorithm (WCPSODA) for route planning, and the study found that the systems work well. Zhang, Rui, et al.[7] use A-star and RRT to create a path to a goal. First, they use A-star to map out a path, and then they use RRT to enhance it. Experimental results show that this hybrid model speeds route planning and reduces average path length compared to RRT alone. Zhang W. et al.[8] introduce the Back Virtual Obstacle Setting Strategy-

School of Mechanical Engineering, Sichuan University Jinjiang College, Meishan 620860, China. ✉email: tangxiaojie@scujj.edu.cn

APF, a new and improved Artificial Potential Field (APF) algorithm. Simulations show that mobile robots can avoid local oscillations and minima with this strategy. Chu Z. et al.[9] improved AUV path planning in unknown environments using Double Deep Q Network-based DRL. Simulations and comparative research prove this strategy works in unfamiliar contexts.

While traditional algorithms offer certain benefits in addressing path-planning challenges, they often exhibit flexibility, fault tolerance, and efficiency limitations. Group intelligence techniques, known for their stability, robustness, and ease of implementation, have proven highly effective in addressing complex optimization issues[10]. The group intelligence algorithm is a bio-heuristic algorithm that models the exchange of information and cooperation between individuals. It achieves efficient group intelligence behaviors by solving the optimal solution of a mathematical model that studies the behavior of simple individuals with information processing capability[11]. Classical group intelligence algorithms like Particle Swarm Optimization (PSO), Ant Colony Optimization (ACO), Artificial Bee Colony (ABC), Differential Evolution (DE), and so on, inspired by animal group behaviors, such as bird flocking, ant colony movement, bee swarming, and cooperation and competition among animals, which have been widely used in mobile robot path planning[12–17]. Recently introduced group intelligence algorithms include the Salp Swarm Algorithm (SSA), Grey Wolf Algorithm (GWO), and Sparrow Search Algorithm (SSA)[18–21] et al. Researchers have enhanced these recently developed population intelligence algorithms to further the investigation of path planning. Luo Y. et al. introduced EWB-GWO, an improved algorithm for GWO, by using the random wandering law in the butterfly optimization algorithm[22]. Simulations show that the EWB-GWO method surpasses the competition in path planning tests for smoothness metrics and optimal path length. Liu et al.[23] improved the sparrow search method's local obstacle avoidance with the sine cosine strategy and dynamic window approach. The revised algorithm performs well. Duan, X. H. et al.[24] upgraded the Salp Swarm Algorithm by introducing population grouping. This unique technique can calculate emergency vehicle rescue routes and evacuation plans. Results show the updated algorithm works.

Introduced by Xue et al.[25] in 2023, the Dung Beetle Optimizer (DBO) algorithm is a novel approach to swarm intelligence that has been successfully implemented to meet a variety of engineering design difficulties. This algorithm has demonstrated its effectiveness, drawing inspiration from the complex social activities seen in groups of dung beetles, such as ball rolling, dancing, foraging, breeding, and stealing. The DBO algorithm was employed by Duan J et al.[26] to determine the ideal hyperparameters for the CNN-LSTM model, which is used to predict air quality. They established the superiority of their paradigm over competing strategies using thorough comparative experiments.

As reported by the No Free Lunch (NFL) theorem[27,28], no method can tackle all optimization issues. The DBO algorithm has several good features, including fast convergence and optimization ability. However, it also has some bad ones, like limited global exploration, vulnerability to local optima, and imbalance between local and global exploitation capabilities. Sun H et al.[29] proposed an enhanced DBO method called IDBO to optimize VMD hyperparameters, improving subway rail corrugation detection. A quantum computing-based, multi-strategy hybrid dung beetle search method (QHDBO) by Zhu F et al.[30] improves convergence speed and optimization accuracy on benchmark functions and real-world engineering difficulties. The application of DBO algorithms to path planning is a relatively new topic. Li L et al.[31] upgraded DBO by using random reverse learning and other methodologies (MSIDBO) and got good path planning simulation results. A directed evolutionary non-dominated sorting dung beetle optimizer with adaptive stochastic ranking (DENSDBO-ASR) was developed by Shen Q et al.[32] to address the difficulty of balancing convergence and diversity in multi-UAV path planning with good results. Lyu L et al.[33] proposed several strategies, including replacing the primitive rolling phase of DBO with a novel global exploration strategy that improves information exchange, and the algorithm achieved the best cost metrics in UAV 3D path planning experiments.

This paper introduces a novel approach termed RCDBO, which enhances the overall exploration power of the DBO algorithm. In a series of comparisons with six different algorithms on various benchmark functions, the RCDBO algorithm proves to be the most effective in speeding up convergence and improving accuracy while also standing out from the crowd. Furthermore, in addressing path planning problems, the RCDBO algorithm showcases superior performance by generating shorter paths amidst various simple and complex environmental maps, outperforming the standard DBO algorithm. Key contributions include:

(1) Leveraging a chaotic map based on Bernoulli to generate initial solutions, enhancing the ergodicity and diversity of the dung beetles' initial distribution and spatial coverage.
(2) A random walk strategy and a kind of crisscross strategy can be employed under certain conditions to perturb dung beetle behaviors, thereby augmenting global search capabilities.
(3) A comprehensive evaluation of the proposed algorithm was conducted using twelve benchmark functions, the CEC2021 test suite, the Wilcoxon rank-sum test, and the Friedman test, with comparative analyses against six advanced swarm intelligence algorithms. Furthermore, the effectiveness of each strategy within the proposed algorithm was assessed using the CEC2021 test suite. A comparative analysis was also performed to examine the balance between exploration and exploitation capabilities in the proposed algorithm relative to the original DBO method.
(4) Implement the method to address path planning issues, with experimental validation providing evidence of its effectiveness.

The original method (DBO)

This section describes Dung Beetle Optimizer (DBO) fundamentals. The algorithm uses Eq. (1) to (8) to update the position of each dung beetle based on their behaviors, like ball-rolling, dancing, breeding, foraging, and stealing. X_i^t represents the position of the ith dung beetle in the tth iteration, $and\ Lb$ and Ub represent the optimizer's lower and upper limits, respectively. This diverse update method allows the algorithm to scan the

search space thoroughly and solve challenging engineering optimization issues. Below are the traits of these five dung beetles.

Ball-rolling behavior Dung bugs use solar navigation to roll dung balls straight. The dung beetle algorithm assumes that light intensity guides the beetle in the search space. This movement updates the beetle's position according to Eq. (1):

$$\begin{gathered} X_i^{t+1} = X_i^t + \alpha \times k \times X_i^{t-1} + b \times \Delta X \\ \Delta X = \left| X_i\left(t\right) - X^w \right| \end{gathered} \tag{1}$$

where t represents the current iteration number; k is the deflection coefficient, with a range of (0, 0.2]; b is a constant, with a range of (0, 1); α is the natural coefficient, taking values of 1 or -1, where 1 indicates no bias, and -1 indicates deviation from the original direction; X^w represents the worst position in the current population; ΔX is used to simulate changes in light intensity.

Dancing behavior Dung beetles dance to reposition themselves and find a new path when rolling dung balls and encountering impediments. Equation (2) updates the dung beetle's position during this process:

$$X_i^{t+1} = X_i^t + \tan\left(\theta\right) \left| X_i^t - X_i^{t-1} \right| \tag{2}$$

where θ represents the deflection angle, with a value range of $[0?\pi]$. When $\theta = 0?\frac{\pi}{2}?\pi$, the dung beetle's position remains unchanged.

Breeding behavior Female dung beetles roll and bury dung balls to produce a safe egg-laying environment. Equation for female dung beetle spawning area:

$$\begin{gathered} Lb^* = \max\left(X^* \times \left(1 - R\right), Lb\right) \\ Ub^* = \min\left(X^* \times \left(1 + R\right), Ub\right) \end{gathered} \tag{3}$$

where Lb^* and Ub^* denote the lower and upper boundaries of spawning, dynamically adjusted with each iteration. X^* indicates the global ideal position of the population. $R = 1 - t/T_{max}$, T_{max} is the maximum number of iterations.

Each female dung beetle produces one brood ball per iteration and deposits eggs in the spawning area. The following equation dynamically adjusts the brood ball's position with spawning area changes during iteration:

$$B_i^{t+1} = X^* + b_1 \times \left(B_i^t - Lb^*\right) + b_2 \times \left(B_i^{t+1} - Ub^*\right) \tag{4}$$

where B_i^{t+1} denotes the position of the ith chick at the tth iteration, and b_1 and b_2 represent two scalar coefficients.

Foraging behavior Equation (5) updates the best feeding regions of mature tiny dung beetles as they emerge from the ground:

$$\begin{gathered} Lb^b = \max\left(X^b \times \left(1 - R\right), Lb\right) \\ Ub^b = \min\left(X^b \times \left(1 + R\right), Ub\right) \end{gathered} \tag{5}$$

where X^b represents the local optimal position of the current population; Lb^b and Ub^b are the lower and upper limits of the optimal foraging area.

The little dung beetle's location update is explained as follows:

$$X_i^{t+1} = X_i^t + C_1 \times \left(X_i^t - Lb^b\right) + C_2 \times \left(X_i^t - Ub^b\right) \tag{6}$$

In this equation, C_1 represents a random number following a normal distribution, and C_2 denotes a random vector between 0 and 1.

Theft Some dung beetles steal dung balls from others. The thief dung beetles update dung balls around the best foraging spots:

$$X_i^{t+1} = X^b + S \times g \times \left(X_i^t - X^*\right) + \left| X_i^t - X^b \right| \tag{7}$$

where g represents a random vector with a size of $1 \times D$ following a normal distribution, and S is a constant.

The proposed method (RCDBO)

Although the original DBO method has impressive optimization abilities and fast convergence, it frequently faces challenges in adequately exploring the global solution space and is prone to getting stuck in local optima, especially when handling complicated problems. To overcome these restrictions and strengthen the DBO's search efficiency, three solutions are suggested: the Bernoulli-based chaotic mapping strategy, the random walk strategy, and the vertical and horizontal crossover strategy. This section will thoroughly examine these tactics.

Bernoulli-based chaotic mapping The DBO approach lacks the ability to guarantee a uniform distribution of beginning positions of individuals in the search area, hence impacting the optimization performance of the algorithm. Chaotic mapping is a method for generating chaotic sequences characterized by high randomness and ergodicity and has been used by many scholars to generate initial solutions for optimization problems, achieving better optimization results than random numbers[34]. There are currently many different types of chaotic mappings, among which the Bernoulli mapping has been proven to be suitable for population initialization[35]. The initial dung beetle population's ergodicity and variety can be increased by using Bernoulli-based chaotic mapping to generate early solutions, enhancing the algorithm's initial solution quality. The following is the formula:

$$x^{t+1} = \begin{cases} \frac{x^t}{1-d}, & x^t \in (0,1-d] \\ \frac{(x^t-1+d)}{d}, & x^t \in (1-d,1) \end{cases} \tag{8}$$

where X^t represents the position of dung beetles in the tth iteration; d is the control parameter, which is used to distinguish the coverage of each partitioned space, $d \in (0,1)$.

To achieve sound control effects, set $d = 0.518$[36]. At this time, the histogram and spatial scatter distribution under chaotic conditions are shown in Fig. 1.

Initializing the population of dung beetles using the Bernoulli chaotic map results in a more randomized and evenly distributed population. This improves global search capabilities, lessens the effect of unequal starting values, and essentially widens the spatial dispersion of the beginning population.

Random walk strategy The rolling ball behavior in the DBO algorithm's initial stage greatly affects its global position search and overall optimization capability. To bolster the DBO method's global optimization prowess and prevent it from Engaging in local minima, the random walk strategy intervenes during the dung beetle's rolling ball behavior phase. the random walk strategy gets involved in the rolling ball behavior phase of the dung beetle. A statistical and mathematical model for trajectories arising from irregular motion is the random walk technique[37]. To do this, a neighboring point of the present solution is chosen at random for comparison. If it performs better than the existing solution, it takes its place as the new focal point. The most advantageous answer is thought to be contained within an N-dimensional sphere if a more favorable value cannot be obtained after N consecutive tries. The center of the sphere is the current optimal solution, and the radius is determined by the current step size. The random walk equation is as follows:

$$X(t) = \{0, c_{cumsum}[2r(1)-1], c_{cumsum}[2r(2)-1], \cdots, c_{cumsum}[2r(n)-1]]\} \tag{9}$$

where $X(t)$ represents a randomized wandering step set; c_{cumsum} is the cumulative sum of wandering steps; n indicates the maximum number of wanderings; $r(t)$ is a random function with t denoting the current number of wandering steps, which is defined as by Eq. (10).

$$r(t) = \begin{cases} 1 & rand > 0.5 \\ 0 & rand \leqslant 0.5 \end{cases} \tag{10}$$

Dung beetles are selected using a roulette wheel method, where each beetle's position affects the boundaries of its random walk. The boundaries of the random walk are defined as follows:

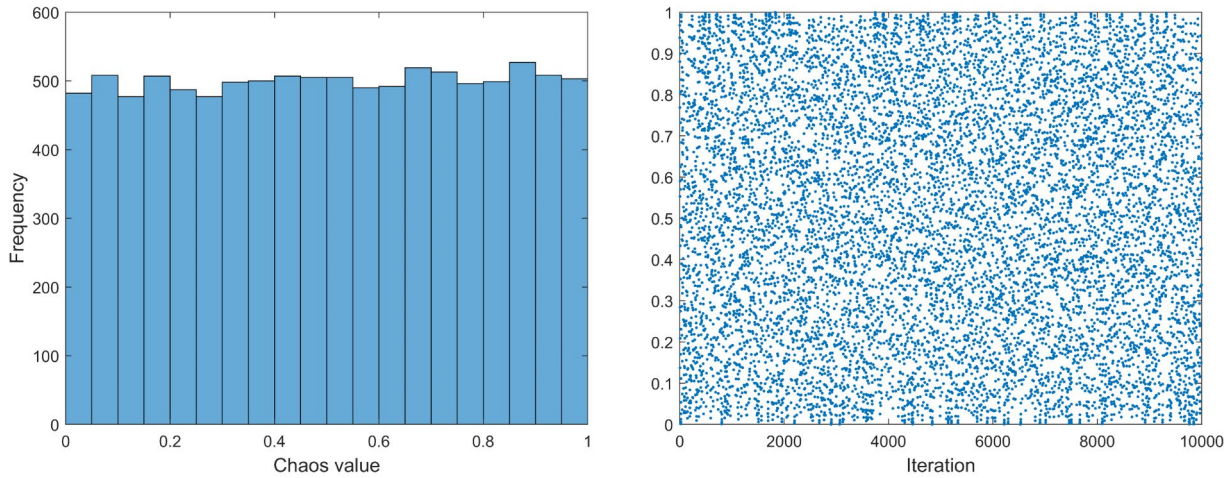


Fig. 1. Bernoulli chaos mapping. Panel **(a)** presents the histogram and Panel **(b)** presents the spatial scatter plot.

$$Ub_d^t = \begin{cases} \frac{P_d^t + Ub_d^t}{I} & rand \geqslant 0.5 \\ \frac{P_d^t - Ub_d^t}{I} & rand < 0.5 \end{cases} \tag{11}$$

$$Lb_d^t = \begin{cases} \frac{P_d^t - Lb_d^t}{I} & rand \geqslant 0.5 \\ \frac{P_d^t + Lb_d^t}{I} & rand < 0.5 \end{cases} \tag{12}$$

where Ub_d^t and Lb_d^t denote the upper and lower bounds of the dth dimension of the tth iteration, respectively; P_d^t denote the value of the dth dimension of the tth iteration; and I is defined as follows:

$$I = \begin{cases} 1 + 10^2 \times \frac{t}{n} & 0.1 \leqslant \frac{t}{n} \leqslant 0.5 \\ 1 + 10^3 \times \frac{t}{n} & 0.5 \leqslant \frac{t}{n} \leqslant 0.75 \\ 1 + 10^4 \times \frac{t}{n} & 0.75 \leqslant \frac{t}{n} \leqslant 0.9 \\ 1 + 10^5 \times \frac{t}{n} & 0.9 \leqslant \frac{t}{n} \leqslant 0.95 \\ 1 + 10^6 \times \frac{t}{n} & 0.95 \leqslant \frac{t}{n} \leqslant 1 \\ 1 & otherwise \end{cases} \tag{13}$$

Since dung beetles' search boundaries are confined within certain limits, it's necessary to normalize Eq. (9) as illustrated in the following equation:

$$P_{i,d}^t = \frac{\left(X_{i,d}^t - a_i\right)\left(Ub_d^t - Lb_d^t\right)}{b_i - a_i} + c_i^t \tag{14}$$

where a_i and b_i denote the minimum and maximum values of the ith dung beetle's random wandering in dimension d, respectively; $X_{i,d}^t$ denotes the position of the ith dung beetle in the tth iteration of the dth dimension.

To prevent the algorithm from becoming more complicated, it is determined that if the best value of the ball-rolling dung beetle remains unchanged for five rounds, the random wandering approach is employed to disrupt the ball-rolling behavior by Eq. (14). According to Eq. 13, it can be seen that during the initial iteration, the range of random wandering is higher, which helps enhance the capacity to search globally. As the iterations go, the range of random wandering is gradually reduced, which helps improve the ability to search locally for the ideal place.

Vertical and horizontal crossover The vertical and horizontal crossover algorithm constitutes an intelligent random search method characterized by a two-group mechanism involving horizontal and vertical crossovers[38]. This approach ensures high convergence precision and swift computation speed. Crossover procedures on two individuals in the same dimension within the population are known as horizontal crossover. This makes cross-learning amongst many dung beetles possible, reducing search blind spots and enhancing the global search capacity. On the other hand, vertical crossover means doing crossover operations on optimal individuals from two different dimensions to increase the diversity of optimal values.

In the horizontal crossover, parental dung beetles within the same dimension are randomly paired, followed by cross-operations among parental individuals in each group to generate offspring[39]. The offspring produced via horizontal crossover undergo fitness comparison with their parents to retain individuals exhibiting higher fitness. The equation for horizontal crossover is as follows:

$$\begin{aligned} X_{inew,d}^t &= a_1 \times X_{i,d}^t + (1 - a_1) \times X_{j,d}^t + b_1 \times \left(X_{i,d}^t - X_{j,d}^t\right) \\ X_{jnew,d}^t &= a_2 \times X_{j,d}^t + (1 - a_2) \times X_{i,d}^t + b_2 \times \left(X_{j,d}^t - X_{i,d}^t\right) \end{aligned} \tag{15}$$

where a_1 and a_2 are arbitrary numbers between 0 and 1, while b_1 and b_2 are between -1 and 1; $X_{i,d}^t$, and $X_{j,d}^t$ represent the parents of generation t and dimension d after combination, respectively; $X_{inew,d}^t$ and $X_{jnew,d}^t$ denote the offspring of generation t and dimension d generated from $X_{i,d}^t$ and $X_{j,d}^t$ through the horizontal crossover.

Below is a numerical illustration that illustrates the horizontal crossing process[40]. Consider the following optimization problem: minimizing the objective function $f(x)$. The problem has two dimensions, and the population size is set to 10. The objective function is defined as:

$$f(x) = 2x_1^2 + x_2^2, \qquad x_1, x_2 \in [-5,5]$$

Initially, the population is generated by randomly selecting 10 points within the range $[-5,5]$, with their specific positions and corresponding objective function values presented in Table 1. Subsequently, random pairings of the population were performed, and Table 2 provides a summary of the outcomes.

The horizontal crossover operation is applied to Parent 1 and Parent 2. Taking the first pair as an example, for the first dimension of the offspring, random coefficient values are generated: $a_1 = 0.0216$, $a_2 = 0.8440$, $b_1 = -0.2656$, $b_2 = 0.5682$, Eq. (11) is then used to compute the first dimension of the offspring as follows:

Values	X_1	X_2	X_3	X_4	X_5	X_6	X_7	X_8	X_9	X_{10}
x_1	0.6	2.5	4.7	1.6	-3.2	1.4	-0.4	0.7	1.8	-1.9
x_2	3.8	-2.1	-1.9	-4.1	3.1	-3.6	3.3	2.2	4.6	-3.7
$f(x)$	15.16	16.91	47.79	21.93	30.09	16.88	11.21	5.82	27.64	20.91

Table 1. Initialization positions and corresponding objective function values.

		Parent 1			Parent 2	
Pairing number	Points	x_1	x_2	Points	x_1	x_2
1	X_1	0.6	3.8	X_6	1.4	-3.6
2	X_{10}	-1.9	-3.7	X_5	-3.2	3.1
3	X_7	-0.4	3.3	X_2	2.5	-2.1
4	X_4	1.6	-4.1	X_3	4.7	-1.9
5	X_8	0.7	2.2	X_9	1.8	4.6

Table 2. Results of randomized pairing.

Values	X^1_{1new}	X^1_{2new}	X^1_{3new}	X^1_{4new}	X^1_{5new}	X^1_{6new}	X^1_{7new}	X^1_{8new}	X^1_{9new}	X^1_{10new}
x_1	1.60	-1.52	-2.51	3.84	-1.73	2.57	1.08	0.42	3.10	-2.28
x_2	0.03	4.94	-3.26	1.12	0.29	5.99	-1.40	0.76	1.54	-3.96
$f(x)$	5.12	29.02	23.23	30.75	6.07	49.09	4.29	0.93	21.59	26.08

Table 3. Offspring generated by lateral crossover and corresponding objective function values.

$$\begin{aligned} X^1_{1new,1} &= a_1 \cdot X^1_{1,1} + (1-a_1) \cdot X^1_{2,1} + b_1 \cdot \left(X^1_{1,1} - X^1_{2,1}\right) \\ &= 0.0216 \times 0.6 + (1-0.0216) \times 1.4 - 0.2656 \times (0.6-1.4) \\ &= 1.60 \end{aligned}$$

$$\begin{aligned} X^1_{2new,1} &= a_2 \cdot X^1_{2,1} + (1-a_2) \cdot X^1_{1,1} + b_2 \cdot \left(X^1_{2,1} - X^1_{1,1}\right) \\ &= 0.8440 \times 1.4 + (1-0.8440) \times 0.6 + 0.5682 \times (1.4-0.6) \\ &= 2.57 \end{aligned}$$

where $X^1_{1new,1}$ denotes the 1st dimension of the child produced by parent 1 and $X^1_{2new,1}$ denotes the 1st dimension of the child produced by parent 2.

Random coefficient values are generated for the second dimension of the offspring: $a_1 = 0.2254$, $a_2 = 0.1735$, $b_1 = 0.2707$, $b_2 = -0.4697$; Eq. (11) is then applied to compute the second dimension of the offspring as follows:

$$\begin{aligned} X^1_{1new,2} &= a_1 \cdot X^1_{1,2} + (1-a_1) \cdot X^1_{2,2} + b_1 \cdot \left(X^1_{1,2} - X^1_{2,2}\right) \\ &= 0.2254 \times 3.8 + (1-0.2254) \times (-3.6) + 0.2707 \times (3.8+3.6) \\ &= 0.03 \end{aligned}$$

$$\begin{aligned} X^1_{2new,2} &= a_2 \cdot X^1_{2,2} + (1-a_2) \cdot X^1_{1,2} + b_2 \cdot \left(X^1_{2,1} - X^1_{1,2}\right) \\ &= 0.1735 \times (-3.6) + (1-0.1735) \times 3.8 - 0.4697 \times (-3.6-3.8) \\ &= 5.99 \end{aligned}$$

where $X^1_{1new,2}$ denotes the 2st dimension of the child produced by parent 1 and $X^1_{2new,2}$ denotes the 2st dimension of the child produced by parent 2.

As a result of the horizontal crossover operation, Parent 1, represented by point X_1, generates Offspring 1 with coordinates (1.60, 0.03), while Parent 2, represented by point X_6, generates Offspring 2 with coordinates (2.57, 5.99). The remaining paired parents produce their offspring in the same manner. The offspring generated by all parents through horizontal crossover, along with their corresponding objective function values, are presented in Table 3. The objective function values of the offspring are compared with those of their respective parents. For each pair, the solution with the lower objective function value is selected for the position update following horizontal crossover. The results of the position updates are summarized in Table 4.

Values	X_1	X_2	X_3	X_4	X_5	X_6	X_7	X_8	X_9	X_{10}
x_1	1.60	2.50	-2.51	1.60	-1.73	1.40	1.08	0.42	3.10	-1.90
x_2	0.03	-2.10	-3.26	-4.10	0.29	-3.60	-1.40	0.76	1.54	-3.70
$f(x)$	5.12	16.91	23.23	21.93	6.07	16.88	4.29	0.93	21.59	20.91

Table 4. Position update results and corresponding objective function values after horizontal crossover.

Next is an introduction to vertical crossover. We present vertical crossover to reduce the possibility that some dung beetle individuals in the population become stuck in local optima in particular dimensions over subsequent iterations. Vertical crossover is a mathematical crossover that occurs across various dimensions of the resulting advantageous solution from horizontal crossover. Each undergoes vertical crossover once, updating only one dimension while keeping the others unchanged[41]. This allows the dimension trapped in local optima to break free while preserving information contained in the other normal dimensions as much as possible. The offspring from vertical crossing are evaluated against their parents, and only individuals with superior fitness are kept. The formula for vertical crossover is as stated:

$$X_{best,d1new}^{t} = a_3 \times X_{best,d1}^{t} + (1 - a_3) \times X_{best,d2}^{t} \tag{16}$$

where a_3 is an arbitrary number between 0 and 1; $X_{best,d1}^{t}$ and $X_{best,d2}^{t}$ represent the global optimal solutions for dimensions $d1$ and $d2$ in the tth generation, respectively; and $X_{best,d1new}^{t}$ denotes the offspring generated by vertical crossover.

Horizontal and vertical crossings occur with a crisscross probability factor P. According to the findings in[38], the choice of the crisscross probability factor P is justified, with the conclusion that it is appropriate to set its value to less than 0.8. Additionally, drawing inspiration from[42] and [43], where improved control performance was achieved by enhancing the control parameters of the gray wolf algorithm using a nonlinear variation strategy, we propose a nonlinear crisscross probability factor. This factor is defined by Eq. (13).

$$P = \frac{2}{3} \times \left[1 - \left(\frac{t}{N}\right)^{1.5}\right] \tag{17}$$

where t denotes the current iteration number, and N is the total number of iterations. If the random number generated in the $t$enth iteration exceeds P, this crossover strategy is employed to perturb the current optimal position of dung beetles. Otherwise, the strategy is skipped.

Figure 2 illustrates that the probability P decreases nonlinearly with the number of iterations. In order to minimize computing expense, the likelihood of adopting the vertical and horizontal crossover approach is decreased during the early stages of the search when population diversity is large. The probability of executing vertical and horizontal crossings rises as the search goes on and population diversity declines, lowering the possibility of convergence to local optima and improving the algorithm's capacity for global search.

Flowchart of RCDBO method Incorporating three enhancement strategies into the DBO algorithm gives rise to the RCDBO algorithm. These strategies involve utilizing Bernoulli chaotic mapping for initializing the dung beetle population, employing the random walk strategy to introduce random perturbations to the rolling ball dung beetles, and applying the vertical and horizontal crossover strategy to perturb the entire dung beetle population. These three enhancement solutions aim to improve the original DBO algorithm's overall search and convergence speed. Figure 3 shows the program flowchart of RCDBO.

Simulation experiment analysis

Experimental design Since no method can tackle all optimization issues, in this section, we design several experiments to illustrate the effectiveness of the proposed RCDBO algorithm, including tests on 12 benchmark test functions (dim = 30\50\100) and the Bias Shift and Rotated Group functions (dim = 20) on the challenging CEC2021 suite. The competitor algorithms chosen are some representative advanced swarm intelligence algorithms, including a classical algorithm: WOA (Whale optimization algorithm, 2016)[44], and two new optimization algorithms that have emerged in the last two years: SABO (Subtraction-average-based Optimizer,2023)[45] and PO (Parrot optimizer, 2024)[46], an improved version of DBO: IDBO (Improved dung beetle optimizer, 2023) mentioned in the previous section, an improved version of a classical algorithm: NCHHO (Nonlinear chaotic Harris hawks optimization, 2021)[47], and the original DBO algorithm. We also performed ablation experiment analysis, exploration, and exploitation behavior analysis, and statistically significant Wilcoxon rank-sum test and Friedman test using the Bias Shift and Rotated Group functions of the CEC2021 suite.

Each algorithm was executed with consistent parameters: the population size was set at 30, the number of exploitation and exploration experiment iterations was set to 1000 to better observe the balance of exploitation and exploration, and 500 for the others. Other relevant parameters were not random numbers, as detailed in Table 5. To ensure robustness, each algorithm underwent 50 independent experiments to mitigate errors due to chance. The simulation software used in the studies was MATLAB R2021b.

Performance on 12 benchmark functions The experimental targets were 12 benchmark functions from Table A1 in Appendix A, with f1 to f7 unimodal and f8 to f12 multimodal. Unimodal functions are used to test the al-

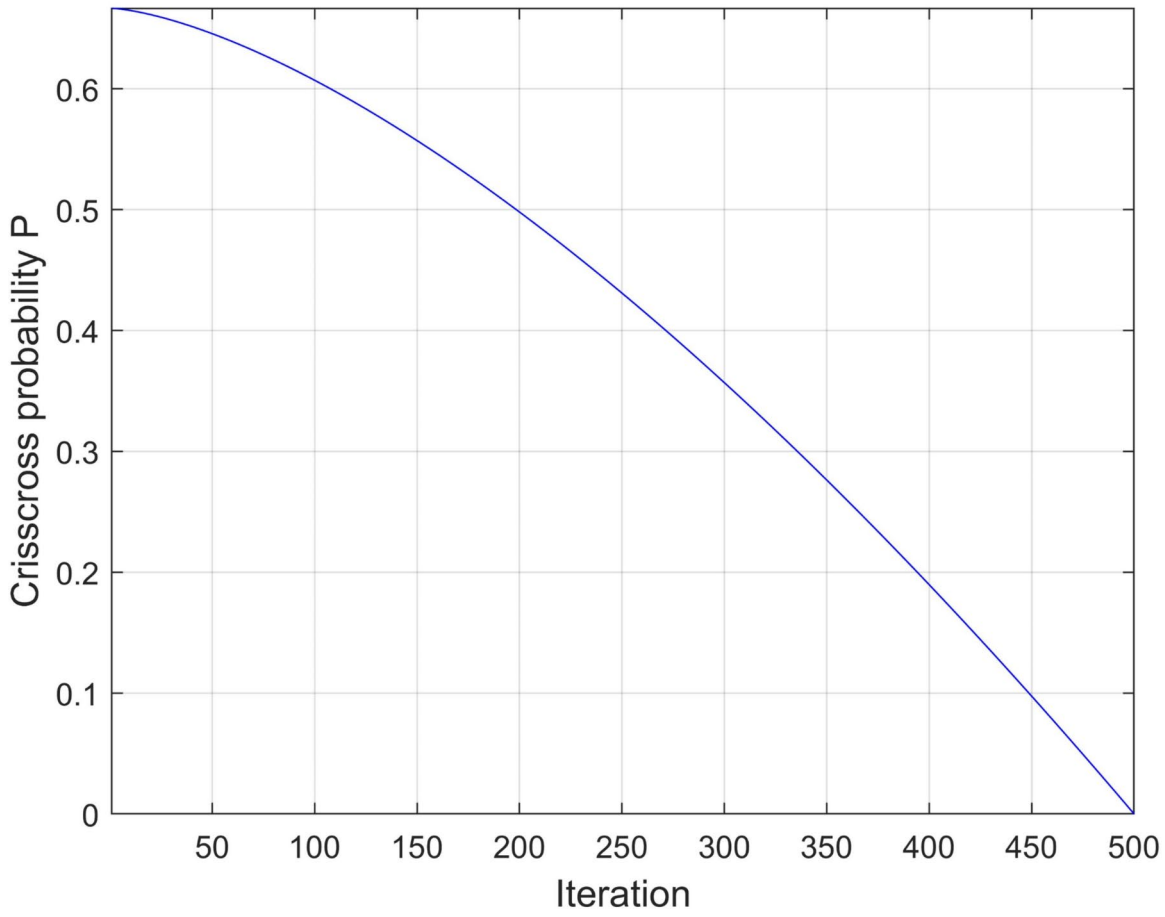


Fig. 2. The curve of variation of the crisscross probability factor P.

gorithm's convergence speed and accuracy. In contrast, multimodal functions are used to examine their capacity to escape local optima and global search due to their many local extremes.

Each of the 12 benchmark functions was subjected to 50 separate testing. The mean, standard deviation, and minimum values for each approach on the twelve benchmark functions with dimensions of 30, 50, and 100 are shown in Table 6. Upon analyzing Table 6, it is evident that the findings for the 30, 50, and 100 dimensions show similarities, with minor changes in the rankings of ideal values. RCDBO achieves a theoretical ideal value of 0 in tests conducted for functions F1 to F4, F6, and F11. For functions F8 to F10, although RCDBO's test data may not consistently yield the theoretical ideal value of 0, it demonstrates substantial advantages over other algorithms, consistently ranking first. Even in the case of function F5, where RCDBO's average value and standard deviation are suboptimal, it outperforms other algorithms in locating the minimum value. Figure 4 illustrates the average fitness iterative convergence comparison curve from these 50 independent experiments at a dimension of 100. As depicted, RCDBO exhibits superior convergence accuracy and speed across all functions except for F5 and F7, where its stability is slightly inferior to that of NCHHO and PO algorithms when testing these two functions, resulting in the average convergence curve not performing best. However, as mentioned above, RCDBO still ranks first in solving the F5 minimum problem. It is evident from the preceding research that RCDBO has a more noteworthy competitive advantage when tackling both single-modal and multimodal problems.

Performance on CEC2021 suite To thoroughly assess the optimization prowess and convergence performance of the RCDBO method, we have enhanced the scope and difficulty of function testing. Intelligent algorithms were employed to tackle the Bias shift and rot group functions from the more challenging CEC2021 test suite, and optimization accuracy and convergence curve comparison experiments were carried out. The CEC2021 test suite comprises ten benchmark functions, wherein the Bias shift and rot group apply transformations of bias, shift, and rotation to these functions to heighten the difficulty of solving while reducing the zero-tendency. Detailed functions are presented in Table A2 of Appendix A, wherein f1 denotes an unimodal function, f2 to f4 signify multimodal functions, f5 to f7 represent mixed functions, and f8 to f10 represent combined functions. The test dimension for all functions is set at the maximum of 20 dimensions.

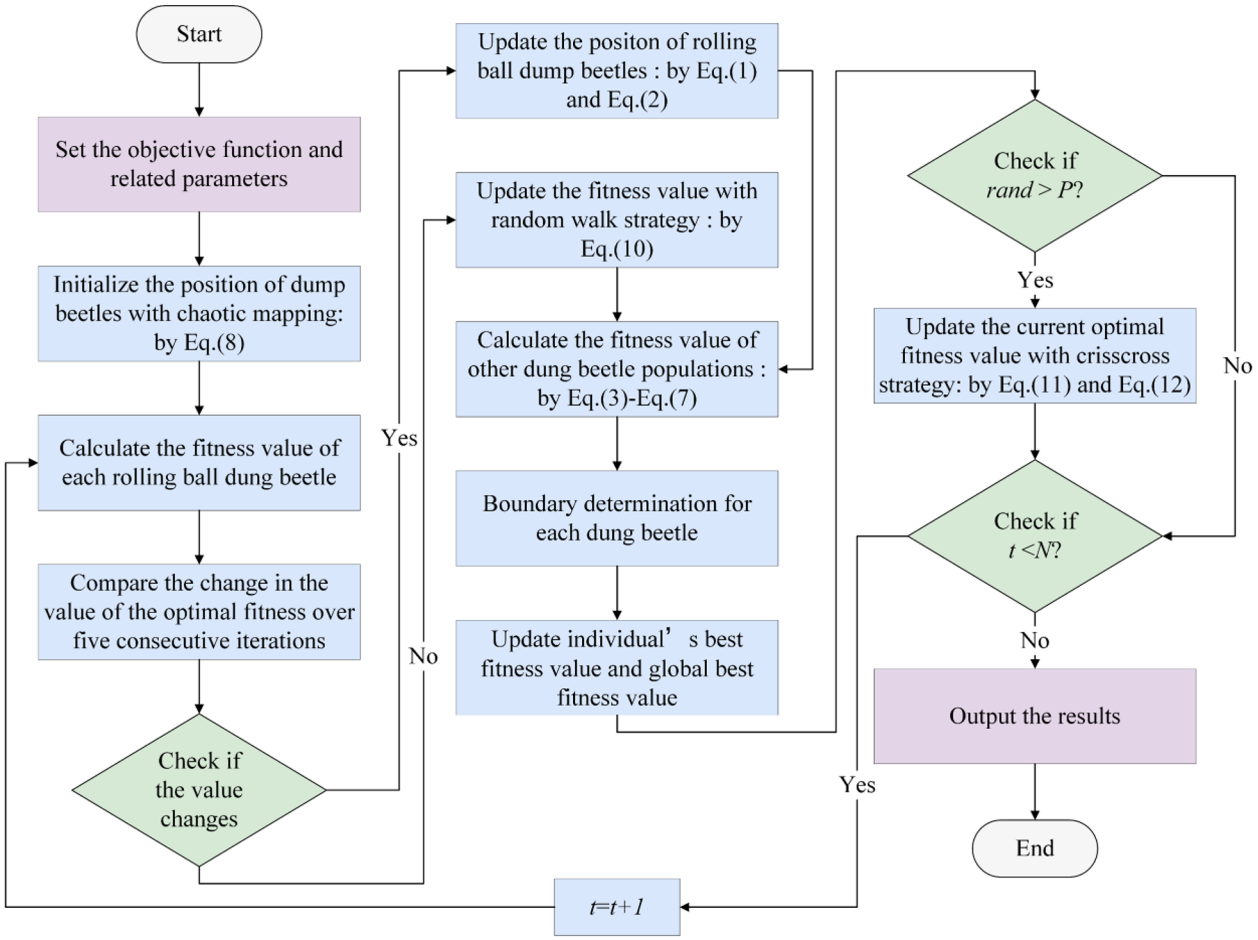


Fig. 3. Flowchart of RCDBO.

Algorithms	Parameters
RCDBO	$e = 0.4$, Disturbance coefficient: $k = 0.1$, Luminous efficacy: $b = 0.3$
DBO	Disruption factor: $k = 0.1$, Luminous efficacy: $b = 0.3$
IDBO	Scale Factor: $k = 10,000$, Weight Factor: $\omega_{min} = 0.2$, $\omega_{max} = 0.9$, Correction coefficient: $a = 0.45$, Proportionality Constant: $k_p = 0.4$
NCHHO	Initial chaotic map parameter: $a = 4$
WOA	Logarithmic spiral shape constant: $b = 1$, Convergence parameter a: Linear reduction from 2 to 0

Table 5. Parameter settings.

Displays the results of the tests, with the optimal values for each item highlighted in bold. The statistics in table 7 clearly show that the technique suggested in this paper achieves good optimization accuracy for most functions. About optimal value, average value, and standard deviation, the RCDBO method outperforms the other six comparison algorithms for functions CEC-1, CEC-4, and CEC-7 to CEC-10. Moreover, the theoretical ideal value for function CEC-6 is successfully identified by the RCDBO algorithm. While the RCDBO algorithm's optimal value for testing function CEC-2 is marginally lower than DBO's, its average value is the best of all the algorithms. For function CEC-3, the RCDBO method has the most optimal value, even if its mean and variance are worse than those of the IDBO and NCHHO algorithms, respectively. As for function CEC-5, the RCDBO algorithm exhibits optimal mean and variance, indicating better stability than the other six algorithms. Furthermore, the average iteration curves in Fig. 5 demonstrate that RCDBO performs slightly worse than the IDBO algorithm in solving the average value of the CEC-3 function. However, it significantly outperforms the other six algorithms regarding accuracy in finding optimal solutions and convergence speed for all other functions. In short, the precision and stability of optimization for the Bias shift and rot group functions of the CEC2021 test suite at dimension 20 are better with the RCDBO algorithm introduced in this work than with the

Fun No.	Name	Dim = 30			Dim = 50			Dim = 100		
		Best	Mean	Std	Best	Mean	Std	Best	Mean	Std
F1	RCDBO	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**
	DBO	5.21E-167	1.16E-107	8.19E-107	5.06E-168	1.09E-106	7.66E-106	1.32E-162	2.28E-107	1.61E-106
	IDBO	0	2.12E-165	0	0	2.73E-163	2.22E-162	0	4.94E-156	2.15E-155
	NCHHO	4.13E-270	3.22E-238	0	2.13E-267	1.08E-239	0	4.91E-268	4.75E-242	0
	SABO	2.85E-200	1.18E-196	0	1.40E-201	1.78E-196	0	7.16E-201	3.82E-197	0
	WOA	3.29E-88	9.71E-74	6.57E-73	4.11E-87	1.57E-69	1.07E-68	9.26E-87	2.39E-71	1.34E-70
	PO	0	2.91E-34	2.06E-33	0	5.79E-35	2.88E-34	0	7.91E-37	5.59E-36
F2	RCDBO	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**
	DBO	5.18E-85	8.05E-55	5.69E-54	1.10E-82	2.61E-56	1.84E-55	1.04E-81	3.02E-54	2.13E-53
	IDBO	6.24E-164	1.77E-92	1.18E-91	5.01E-170	5.92E-90	2.61E-89	2.20E-171	1.50E-88	9.45E-88
	NCHHO	9.84E-138	2.57E-119	1.82E-118	2.08E-135	4.48E-123	1.71E-122	4.01E-138	5.61E-123	2.57E-122
	SABO	4.43E-113	3.63E-111	4.82E-111	2.25E-114	2.12E-112	3.73E-112	1.07E-117	5.86E-115	1.48E-114
	WOA	1.31E-57	3.10E-51	9.59E-51	1.19E-59	2.01E-50	1.27E-49	5.45E-58	3.93E-49	1.87E-48
	PO	**0**	1.09E-16	4.94E-16	3.26E-55	2.69E-18	8.82E-18	**0**	5.03E-14	3.52E-13
F3	RCDBO	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**
	DBO	3.27E-144	4.82E-58	3.41E-57	3.11E-151	2.70E-15	1.91E-14	3.94E-154	3.20E-03	2.26E-02
	IDBO	**0**	1.49E-123	9.59E-123	**0**	1.22E-117	8.49E-117	1.13E-163	2.16E-110	1.52E-109
	NCHHO	3.58E-245	4.44E-215	**0**	3.85E-258	5.55E-210	**0**	4.18E-240	2.02E-201	**0**
	SABO	6.69E-86	2.35E-37	1.66E-36	9.06E-72	4.20E-26	2.44E-25	2.25E-63	1.11E-06	7.83E-06
	WOA	8.23E+03	4.06E+04	1.17E+04	1.15E+05	1.94E+05	4,05E+04	5.03E+05	1.07E+06	3.05E+05
	PO	**0**	9.32E-31	4.09E-30	**0**	3.22E-29	1.37E-28	6.80E-86	1.93E-24	1.32E-23
F4	RCDBO	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**
	DBO	5.72E-81	3.91E-51	2.77E-50	2.32E-81	5.42E-52	3.03E-51	1.38E-75	1.42E-44	9.95E-44
	IDBO	1.09E-176	1.45E-71	1.02E-70	4.20E-170	5.80E-64	3.96E-63	2.51E-168	2.81E-61	1.34E-60
	NCHHO	2.3E-133	1.07E-119	6.44E-119	9.75E-130	3.72E-117	2.62E-116	2.24E-132	1.33E-117	8.95E-117
	SABO	1.37E-79	5.64E-77	9.51E-77	8.65E-77	3.15E-75	7.68E-75	5.15E-75	1.05E-73	1.59E-73
	WOA	1.71E+02	5.67E+01	2.56E+01	1.24E+00	6.97E+01	2.30E+01	1.80E+01	7.62E+01	2.29E+01
	PO	2.14E-48	5.78E-18	2.88E-17	**0**	1.99E-18	1.40E-17	**0**	2.93E-19	1.68E-18
F5	RCDBO	1.66E-07	2.02E+00	6.92E+00	3.08E-07	1.82+E00	9.02E+00	6.10E-11	9.54E+00	2.89E+01
	DBO	2.53E+01	2.58E+01	1.97E-01	4.54E+01	4.63E+01	7.64E-01	9.60E+01	9.71E+01	7.37E-01
	IDBO	1.49E-03	1.37E+01	1.31E+01	4.36E-03	2.91E+01	2.24E+01	1.15E-02	4.38E+01	4.74E+01
	NCHHO	2.08E-07	2.06E-03	3.96E-03	4.24E-06	3.02E-03	7.00E-03	3.68E-06	9.07E-03	2.11E-02
	SABO	2.76E+01	2.84E+01	3.93E-01	4.79E+01	4.86E+01	2.78E-01	9.80E+01	9.86E+01	1.86E-01
	WOA	2.13E+00	2.75E+01	3.70E+00	4.75E+01	4.83E+01	3.53E-01	9.72E+01	9.82E+01	2.31E-01
	PO	2.32E-07	3.12E-02	5.36E-02	2.08E-06	5.02E-02	8.41E-02	1.60E-06	1.70E-01	3.53E-01
F6	RCDBO	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**4.25E-20**	**2.17E-19**
	DBO	3.41E-06	7.01E-04	2.01E-03	2.05E-02	2.43E-01	1.97E-01	2.69E+00	4.72E+00	8.78E-01
	IDBO	1.92E-08	1.96E-05	5.84E-05	1.29E-04	1.42E-02	4.60E-02	3.78E-03	9.63E-02	1.12E-01
	NCHHO	1.75E-08	2.84E-05	1.17E-04	7.70E-10	3.17E-05	6.92E-05	1.72E-08	4.23E-05	7.13E-05
	SABO	1.32E+00	2.53E+00	5.58E-01	4.41E+00	6.30E+00	7.47E-01	1.34E+01	1.62E+01	1.17E+00
	WOA	5.69E-02	3.95E-01	2.85E-01	2.99E-01	1.28E+00	3.93E-01	1.62E+00	3.89E+00	1.18E+00
	PO	9.12E-13	1.66E-04	1.96E-04	9.90E-08	4.92E-04	7.63E-04	3.65E-09	6.89E-04	7.89E-04
F7	RCDBO	4.61E-06	1.13E-04	9.13E-05	2.80E-07	1.22E-04	1.04E-04	3.84E-06	1.57E-04	1.42E-04
	DBO	5.71E-05	1.17E-03	8.44E-04	5.37E-05	1.05E-03	8.51E-04	2.64E-05	1.20E-03	9.76E-04
	IDBO	2.29E-05	2.51E-04	1.81E-04	5.74E-06	2.16E-04	1.81E-04	2.12E-05	2.33E-04	1.70E-04
	NCHHO	3.02E-07	1.28E-04	1.21E-04	7.73E-06	1.39E-04	1.30E-04	**2.37E-06**	1.26E-04	1.11E-04
	SABO	**1.44E-07**	1.09E-04	9.88E-05	1.14E-05	1.21E-04	9.18E-05	8.29E-06	1.56E-04	1.45E-04
	WOA	7.01E-05	3.59E-03	3.75E-03	3.75E-05	3.54E-03	3.79E-03	3.75E-05	3.79E-03	5.41E-03
	PO	3.98E-07	**2.24E-05**	**1.70E-05**	**1.37E-07**	**2.50E-05**	**2.04E-05**	4.93E-06	**2.98E-05**	**2.03E-05**

Continued

F8	RCDBO	**1.35E-32**	**1.35E-32**	**2.76E-48**	**1.35E-32**	**1.33E-30**	**8.38E-30**	**1.35E-32**	**5.50E-19**	**3.88E-18**
	DBO	1.05E-03	5.76E-01	4.66E-01	5.15E-01	2.83E+00	7.28E-01	3.82E+00	8.62E+00	9.02E-01
	IDBO	1.36E-09	2.00E-03	4.29E-03	1.40E-05	4.46E-03	6.93E-03	3.69E-04	3.61E-02	4.39E-02
	NCHHO	4.51E-08	2.61E-05	5.15E-05	3.13E-08	2.29E-05	3.00E-05	1.71E-08	7.32E-05	1.78E-04
	SABO	1.44E+00	2.72E+00	4.96E-01	4.94E+00	4.98E+00	1.16E-02	9.93E+00	9.96E+00	9.90E-03
	WOA	1.84E-01	5.46E-01	2.00E-01	3.73E-01	1.13E+00	4.46E-01	8.41E-01	2.62E+00	9.93E-01
	PO	8.61E-10	5.29E-05	7.49E-05	9.80E-08	9.26E-05	1.59E-04	2.85E-09	1.12E-04	2.47E-04
F9	RCDBO	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**
	DBO	**0**	2.49E+00	9.29E+00	**0**	1.67E+00	8.50E+00	0	1.31E+00	9.29E+00
	IDBO	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**
	NCHHO	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**
	SABO	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**
	WOA	**0**	2.27E-15	1.13E-14	**0**	6.82E-15	2.73E-14	**0**	2.27E-15	1.61E-14
	PO	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**
F10	RCDBO	**8.88E-16**	**8.88E-16**	**0**	**8.88E-16**	**8.88E-16**	**0**	**8.88E-16**	**8.88E-16**	**0**
	DBO	**8.88E-16**	**8.88E-16**	**0**	**8.88E-16**	1.03E-15	7.03E-16	**8.88E-16**	9.59E-16	5.02E-16
	IDBO	**8.88E-16**	**8.88E-16**	**0**	**8.88E-16**	**8.88E-16**	**0**	**8.88E-16**	**8.88E-16**	**0**
	NCHHO	**8.88E-16**	**8.88E-16**	**0**	**8.88E-16**	**8.88E-16**	**0**	**8.88E-16**	**8.88E-16**	**0**
	SABO	4.44E-15	4.51E-15	5.02E-16	4.44E-15	4.44E-15	**0**	4.44E-15	4.44E-15	**0**
	WOA	**8.88E-16**	4.73E-15	2.67E-15	**8.88E-16**	5.01E-15	2.08E-15	**8.88E-16**	4.51E-15	2.73E-15
	PO	**8.88E-16**	9.59E-16	5.02E-16	**8.88E-16**	1.10E-15	8.52E-16	**8.88E-16**	9.59E-16	5.02E-16
F11	RCDBO	**0**	**0**	**0**	**0**	**00**	**0**	**0**	**0**	**0**
	DBO	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**
	IDBO	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**
	NCHHO	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**
	SABO	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**
	WOA	**0**	4.44E-18	2.20E-17	**0**	9.05E-03	4.52E-02	**0**	2.22E-18	1.57E-17
	PO	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**
F12	RCDBO	**1.57E-32**	**1.57E-32**	**0**	**9.42E-33**	**2.09E-31**	**1.25E-30**	**4.71E-33**	**5.04E-20**	**3.43E-19**
	DBO	3.20E-07	4.04E-03	1.93E-02	3.53E-04	4.21E-03	4.89E-03	2.98E-02	6.49E-02	2.85E-02
	IDBO	3.32E-10	2.29E-05	1.29E-04	2.53E-06	3.43E-04	1.32E-03	2.68E-05	1.10E-03	1.05E-03
	NCHHO	2.31E-10	2.76E-06	4.45E-06	4.38E-10	9.93E-07	1.85E-06	1.45E-09	8.01E-07	2.42E-06
	SABO	6.03E-02	2.43E-01	9.76E-02	1.95E-01	3.93E-01	1.29E-01	4.18E-01	5.96E-01	8.44E-02
	WOA	2.20E-03	2.21E-02	1.88E-02	7.88E-03	3.21E-02	2.51E-02	1.48E-02	4.53E-02	1.84E-02
	PO	7.95E-13	1.60E-05	1.61E-05	1.47E-09	9.69E-06	1.13E-05	4.57E-11	1.27E-05	2.86E-05

Table 6. 12 benchmark test function results (dim = 30/50/100).

other six comparative algorithms. This shows that the RCDBO algorithm still optimizes the harder CEC2021 test suite.

Wilcoxon rank-sum test and friedman test Statistical tests employing the Wilcoxon rank-sum test and the Friedman test were performed on the CEC2021 test suite to further analyze the differences and superiority of the RCDBO algorithm with other algorithms[48].

The Wilcoxon Rank-Sum Test calculates the p-value of the statistical results to ascertain their significance. The p-value can be computed by comparing the RCDBO algorithm with the other six algorithms; if $p < 0.05$, there is a noteworthy difference between the two methods. Table 8 displays the Wilcoxon Rank-Sum Test findings; p-values above 0.05 are highlighted. The number of cases in which the RCDBO algorithm performs better than, equal to, or worse than the other algorithms is shown in the last row of the table by the symbols +/=/-. The RCDBO algorithm's optimization performance is significantly different from the other six algorithms throughout the ten benchmark functions and better than them, as seen by the table's majority of p-values below 0.05.

The Friedman test ranks the algorithms based on the calculated average ranks, where higher ranks indicate better algorithm performance, thereby comparing the superiority of different algorithms. The Friedman ranking results are shown in Fig. 6, revealing that except for the CEC-3 function, where the RCDBO algorithm ranks second, it ranks first in all other functions. In brief, the RCDBO algorithm exhibits exceptional optimization performance in the Wilcoxon rank-Sum and Friedman tests.

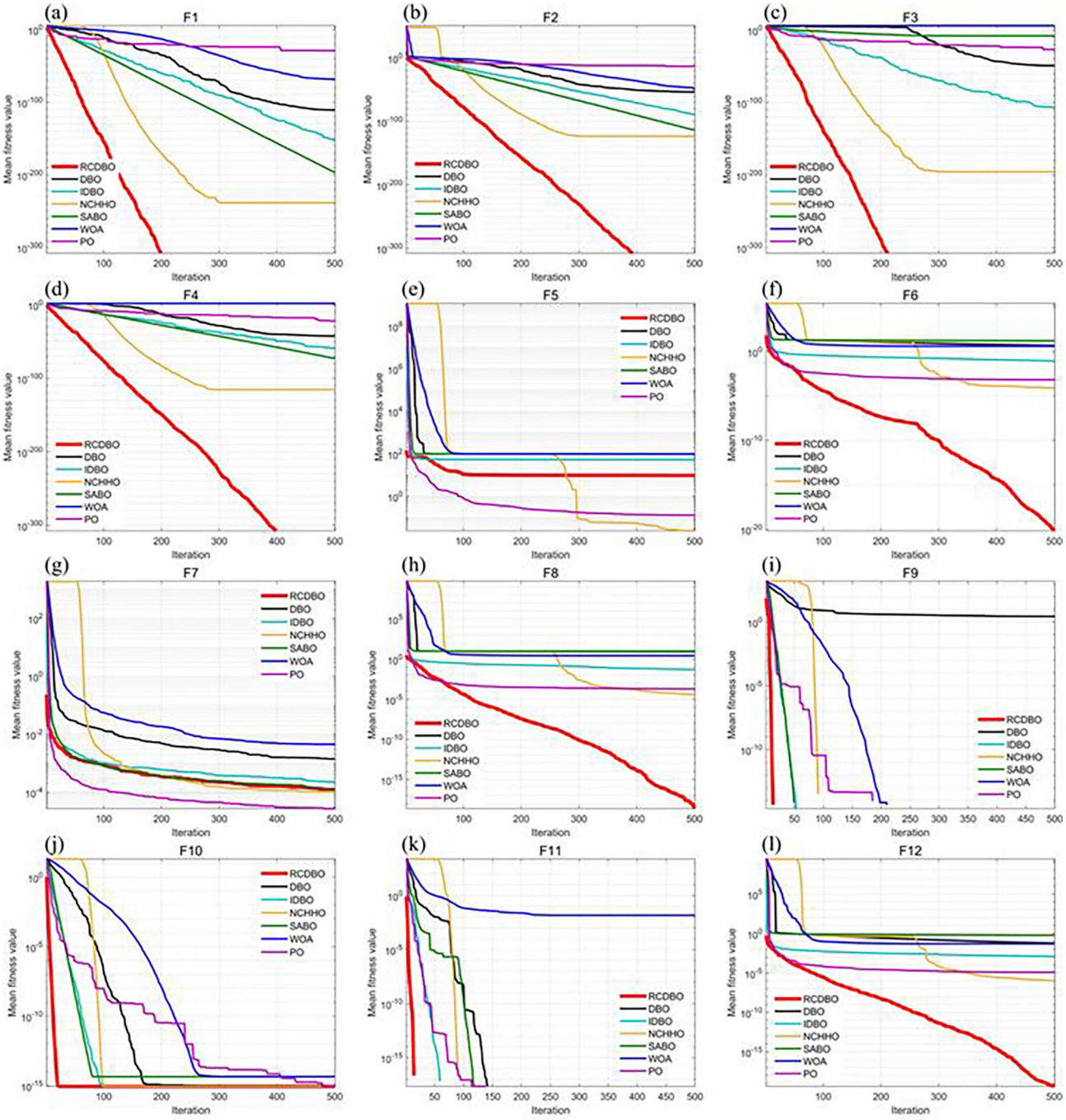


Fig. 4. Average iterative curves on benchmark test function (dim = 100).

Ablation experiments RCDBO1, RCDBO2, and RCDBO3 were generated by applying the DBO algorithm in conjunction with Bernoulli-based chaotic mapping, the random walk, and vertical and horizontal crossover strategies, respectively. To assess the effectiveness and feasibility of these strategies, the RCDBO algorithm was compared against RCDBO1, RCDBO2, RCDBO3, and the original DBO algorithm on the Bias Shift and Rotated Group functions from the CEC2021 test suite. Table 9 displays the comparative results, with the ideal values indicated in bold.

The experimental results show that each strategy plays its role. However, RCDBO performs the best, indicating that the chaotic mapping makes the initial values more homogeneous and enlarges the population search space. The random wandering method and the vertical and horizontal crossover strategies augment the algorithm's global search capacity through the introduction of disturbances.

Fun No.	Index	RCDBO	DBO	IDBO	NCHHO	SABO	WOA	PO
CEC-1	Best	**1.01E+02**	5.89E+03	1.23E+02	1.20E+10	1.67E+08	2.74E+08	3.17E+08
	Mean	**4.34E+03**	2.99E+07	2.17E+07	2.50E+10	4.87E+09	1.43E+09	3.27E+09
	Std	**4.35E+03**	2.76E+07	3.38E+07	4.60E+09	2.92E+09	7.40E+08	1.77E+09
CEC-2	Best	2.02E+03	**1.92E+03**	2.13E+03	4.39E+03	5.31E+03	2.56E+03	3.01E+03
	Mean	**3.04E+03**	3.59E+03	3.31E+03	5.46E+03	5.82E+03	4.43E+03	4.09E+03
	Std	5.08E+02	6.56E+02	9.02E+02	4.24E+02	**2.48E+02**	6.27E+02	5.25E+02
CEC-3	Best	**7.56E+02**	7.63E+02	7.72E+02	9.56E+02	8.53E+02	8.88E+02	8.35E+02
	Mean	8.23E+02	8.34E+02	**8.20E+02**	1.01E+03	9.02E+02	9.64E+02	9.23E+02
	Std	3.74E+01	4.08E+01	3.20E+01	**2.45E+01**	3.34E+01	4.69E+01	3.91E+01
CEC-4	Best	**1.90E+03**	1.91E+03	**1.90E+03**	9.93E+03	1.92E+03	1.92E+03	1.92E+03
	Mean	**1.91E+03**	1.95E+03	1.92E+03	3.33E+05	1.25E+04	2.57E+03	2.10E+03
	Std	**4.50E+00**	1.01E+02	8.37E+00	1.75E+05	1.52E+04	9.20E+02	4.50E+02
CEC-5	Best	4.66E+04	**3.19E+04**	8.72E+04	5.71E+05	8.70E+04	4.03E+04	7.77E+04
	Mean	**2.79E+05**	1.06E+06	5.26E+05	3.55E+06	8.75E+05	2.85E+06	1.02E+06
	Std	**1.46E+05**	1.14E+06	6.17E+05	2.20E+06	6.11E+05	2.49E+06	7.04E+05
CEC-6	Best	**1.63E+03**	1.72E+03	1.69E+03	2.41E+03	2.29E+03	2.17E+03	1.94E+03
	Mean	**2.06E+03**	2.22E+03	2.16E+03	3.27E+03	3.00E+03	2.69E+03	2.35E+03
	Std	2.17E+02	2.43E+02	2.63E+02	3.65E+02	2.67E+02	2.82E+02	**2.15E+02**
CEC-7	Best	**3.67E+03**	6.58E+03	1.56E+04	2.12E+05	8.97E+04	2.97E+04	1.17E+04
	Mean	**7.41E+04**	4.94E+05	3.98E+05	3.86E+06	6.84E+05	1.78E+06	3.62E+05
	Std	**7.29E+04**	4.12E+05	3.89E+05	3.59E+06	6.40E+05	1.59E+06	4.09E+05
CEC-8	Best	**2.30E+03**	**2.30E+03**	**2.30E+03**	3.88E+03	2.43E+03	2.36E+03	2.34E+03
	Mean	**2.30E+03**	2.61E+03	2.32E+03	6.13E+03	2.81E+03	4.32E+03	2.56E+03
	Std	**1.42E+00**	8.38E+02	1.28E+01	8.56E+02	3.32E+02	1.90E+03	2.78E+02
CEC-9	Best	**2.83E+03**	2.88E+03	2.87E+03	3.13E+03	2.91E+03	2.94E+03	2.88E+03
	Mean	**2.91E+03**	2.99E+03	2.96E+03	3.39E+03	3.03E+03	3.04E+03	2.97E+03
	Std	**3.70E+01**	5.44E+01	5.12E+01	1.60E+02	6.49E+01	5.99E+01	5.38E+01
CEC-10	Best	**2.90E+03**	2.91E+03	2.91E+03	3.76E+03	3.03E+03	2.98E+03	2.97E+03
	Mean	**2.97E+03**	2.99E+03	2.98E+03	5.02E+03	3.24E+03	3.12E+03	3.11E+03
	Std	**3.31E+01**	7.69E+01	4.19E+01	7.70E+02	2.12E+02	7.37E+01	9.42E+01

Table 7. CEC2021 test function results (dim = 20).

Exploitation and exploration analysis Exploitation and exploration experiments can be used to assess the problem of balancing the global search capability and the local exploitation capability of various heuristic algorithms[49,50]. Exploitation corresponds to local search, meaning that when the population is convergent, the variance decreases, and better solutions are searched for by exploitation in the aggregated region of the current population of individuals. Exploration corresponds to global search, meaning that when the population is dispersed in the search environment, the likelihood of finding the globally optimal location is maximized by exploring to visit invisible neighborhoods in the search environment. The mean value of diversity in each dimension is first calculated using Eq. (14), and then the percentage of exploitation and exploration is calculated using Eq. (15) and Eq. (16).

$$Div = \frac{1}{D}\sum_{j=1}^{D}\frac{1}{N}\left(\sum_{i=1}^{N}\left|median\left(x_j\right) - x_{ij}\right|\right) \tag{14}$$

$$Exploration\% = \frac{Div}{Div_{max}} \times 100\% \tag{15}$$

$$Exploitation\% = \frac{|Div - Div_{max}|}{Div_{max}} \times 100\%$$

where x_i is the jth dimensional position of the ith dung beetle, and $median\left(x_{ij}\right)$ is the median position of all dung beetles in the jth dimension. Div denotes the diversity value of the population, and Div_{max} denotes the maximum diversity value in the iteration.

In this experiment, the exploration and exploitation capabilities of the DBO algorithm and the proposed RCDBO algorithm are comparatively analyzed when applied to the challenging Bias Shift and Rotated Group functions (dim = 20) from the CEC2021 test suite. The experiment was carried out 50 times, and the results regarding exploration and exploitation percentages are presented in Fig. 7. The RCDBO and DBO algorithms exhibit a dominance of exploration in the early iterations, with the solution set maintaining a high degree of

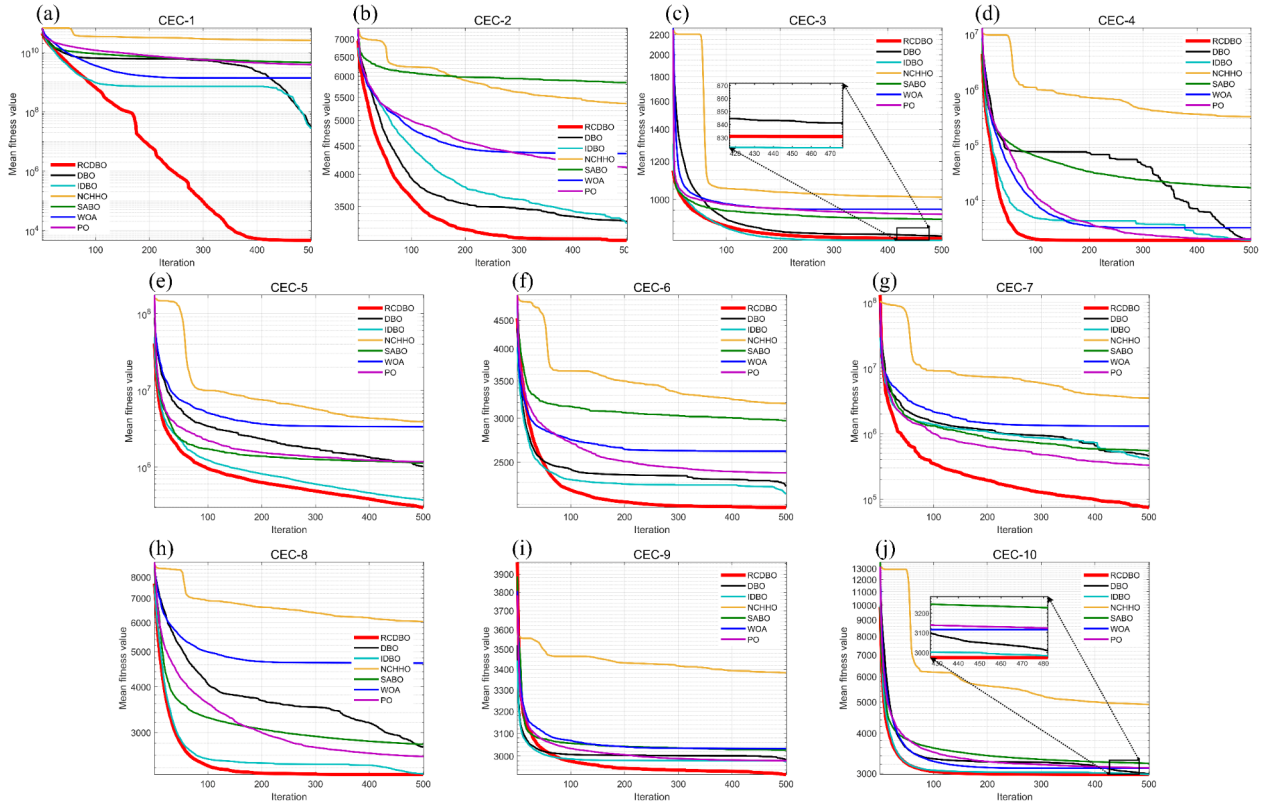


Fig. 5. Average iterative curves on CEC2021 test function (dim = 20).

Fun No.	DBO	IDBO	NCHHO	SABO	WOA	PO
CEC-1	5.02E-17	3.19E-07	7.07E-18	7.07E-18	7.07E-18	7.07E-18
CEC-2	1.47E-05	*4.26E-01*	7.07E-18	7.07E-18	3.02E-15	4.90E-13
CEC-3	*2.06E-01*	*7.80E-01*	7.07E-18	1.10E-13	1.54E-17	1.17E-15
CEC-4	5.29E-14	1.98E-07	7.07E-18	7.97E-18	7.97E-18	1.54E-17
CEC-5	2.89E-10	8.71E-03	8.99E-18	2.05E-09	2.56E-11	1.12E-10
CEC-6	7.97E-04	*6.32E-02*	1.01E-17	1.45E-17	7.49E-16	4.93E-09
CEC-7	7.80E-12	2.22E-11	8.46E-18	4.50E-16	3.38E-15	4.36E-09
CEC-8	2.20E-17	1.29E-10	7.07E-18	7.07E-18	7.07E-18	7.07E-18
CEC-9	5.56E-12	1.93E-06	7.07E-18	3.38E-15	2.54E-16	2.30E-08
CEC-10	*7.12E-01*	*1.65E-01*	7.07E-18	7.07E-18	3.53E-17	9.92E-16
+/-/=	8/2/0	6/4/0	10/0/0	10/0/0	10/0/0	10/0/0

Table 8. Wilcoxon rank-sum test results.

dispersion in the search space. As the evolution progresses, the population gradually converges towards the optimal solution, with the entire solution set becoming concentrated near the global optimum.

Moreover, except three hybrid functions (f5, f6, and f7), where the difference in the balance between exploration and exploitation for RCDBO and DBO is not pronounced, the RCDBO algorithm demonstrates significantly superior performance compared to DBO on other unimodal, multimodal, and composite functions, which is more oriented towards exploration in its early stages, and more oriented towards exploitation in its later stages when the population is gradually approaching the optimal solution. The chaotic initialization strategy, based on the Bernoulli distribution, and the random walk strategy incorporated into the initial ball-rolling behavior of the Dung Beetle enhance the initial exploration capabilities of RCDBO, leading to faster convergence. Meanwhile, the introduction of vertical and horizontal crossover strategies better balances the exploration and exploitation capabilities of the algorithm, helping to avoid local optima and facilitating the discovery of the global optimal solution.

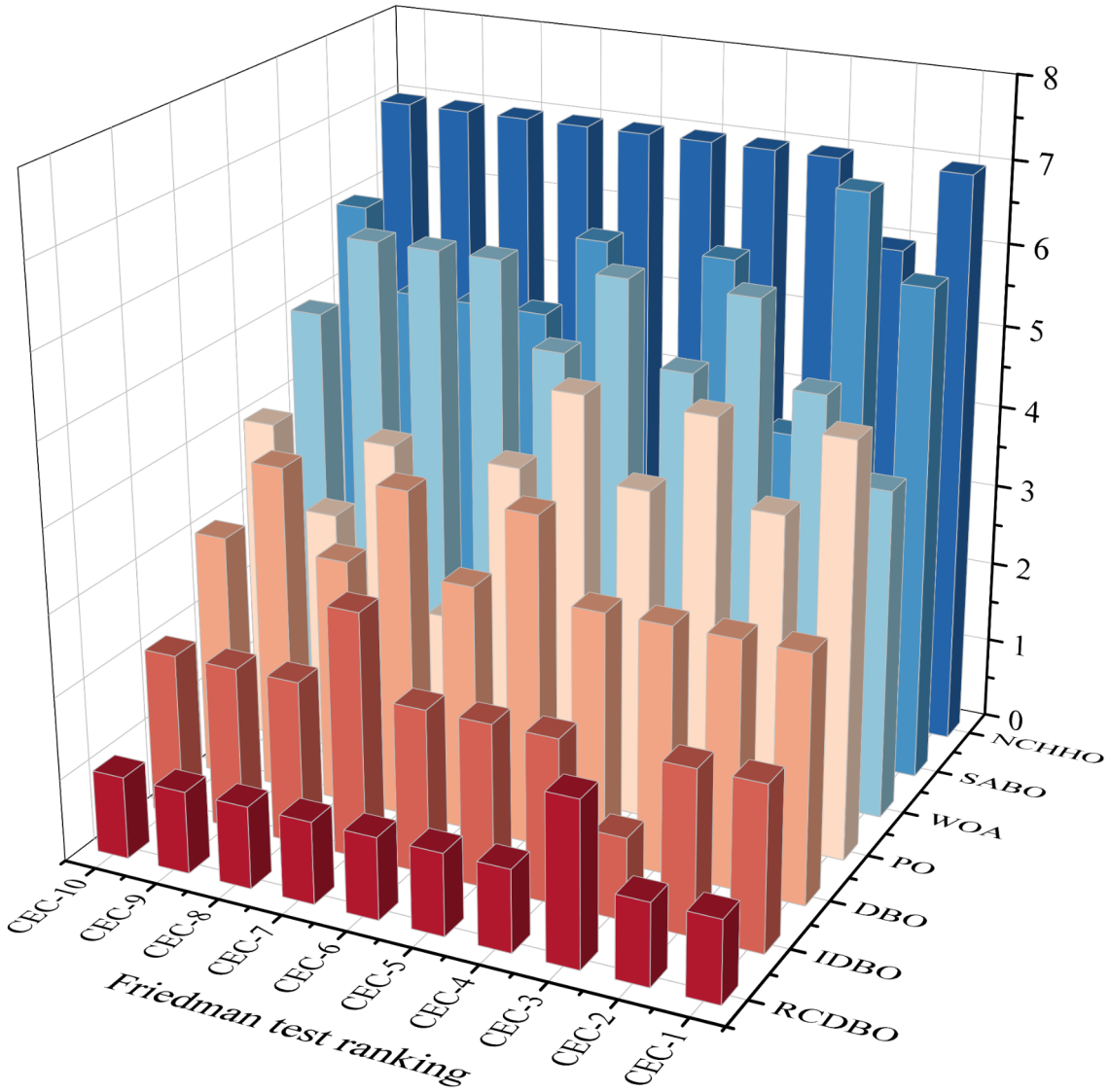


Fig. 6. The ranking results of Friedman test.

Path planning simulation and analysis

Experimental model Moving vehicles is considered a mass point in the experimental model for path planning problem solutions. A grid-based approach is utilized to establish an environmental map, where the collection of grids represents the actual map[51]. This effectively binarizes all objects in the scene, with obstacles represented as one and non-obstacles as 0. The algorithm seeks to create the best route for the moving car on the map, free of obstacles from the starting point to the destination. Reduced overall path length is the optimization goal in path planning, which improves vehicle mobility efficiency and lowers energy consumption. The following is, therefore the fitness function expressed:

$$d = \sum_{k=1}^{n} \sqrt{(x_{k+1} - x_k)^2 + (y_{k+1} - y_k)^2} \tag{15}$$

where (x_k, y_k) denotes the coordinates of the kth node, d is the length of the route taken by the vehicle from the point of origin to the final destination, and n is the number of intermediate points along the trip.

Simulation and analysis This study presents the RCDBO algorithm as a solution to vehicle route planning difficulties. We conducted an experimental comparative analysis between RCDBO and the original DBO algorithm to prove its efficacy and practicality. This aimed to examine RCDBO's optimization performance in such scenarios.

Fun No.	Index	RCDBO	RCDBO1	RCDBO2	RCDBO3	DBO
CEC-1	Best	**1.00E+02**	2.02E+02	3.62E+03	1.09E+02	1.07E+03
	Mean	5.22E+03	3.31E+07	1.54E+05	**4.99E+03**	3.70E+07
	Std	**4.49E+03**	4.56E+07	5.48E+05	4.67E+03	4.37E+07
CEC-2	Best	1.92E+03	1.99E+03	**1.77E+03**	2.15E+03	2.03E+03
	Mean	**3.03E+03**	3.41E+03	3.31E+03	3.12E+03	3.57E+03
	Std	5.38E+02	6.99E+02	6.85E+02	**4.78E+02**	6.99E+02
CEC-3	Best	**7.54E+02**	7.70E+02	7.87E+02	7.72E+02	7.77E+02
	Mean	**8.36E+02**	8.39E+02	8.85E+02	8.49E+02	8.45E+02
	Std	5.81E+01	**4.01E+01**	4.79E+01	4.02E+01	4.45E+01
CEC-4	Best	**1.90E+03**	1.91E+03	**1.90E+03**	**1.90E+03**	1.91E+03
	Mean	**1.91E+03**	1.95E+03	1.92E+03	**1.91E+03**	1.94E+03
	Std	**3.84E+00**	2.12E+02	9.78E+00	6.56E+00	7.46E+01
CEC-5	Best	**2.19E+04**	1.13E+05	1.19E+05	5.91E+04	7.33E+04
	Mean	**2.69E+05**	4.17E+05	7.17E+05	3.17E+05	9.85E+05
	Std	**1.37E+05**	3.66E+05	8.83E+05	1.70E+05	9.01E+05
CEC-6	Best	**1.70E+03**	1.75E+03	1.74E+03	1.72E+03	1.72E+03
	Mean	2.09E+03	2.25E+03	2.15E+03	**1.97E+03**	2.23E+03
	Std	1.98E+02	2.56E+02	2.41E+02	**1.82E+02**	3.08E+02
CEC-7	Best	**4.81E+03**	7.26E+03	1.40E+04	7.76E+03	1.22E+04
	Mean	**1.02E+05**	2.17E+05	3.16E+05	1.49E+05	5.03E+05
	Std	**1.02E+05**	3.43E+05	2.85E+05	1.36E+05	5.91E+05
CEC-8	Best	**2.30E+03**	**2.30E+03**	**2.30E+03**	**2.30E+03**	**2.30E+03**
	Mean	**2.30E+03**	2.32E+03	2.71E+03	**2.30E+03**	2.56E+03
	Std	**1.26E+00**	8.36E+00	9.87E+02	1.90E+00	7.40E+02
CEC-9	Best	**2.84E+03**	2.90E+03	2.88E+03	**2.84E+03**	2.88E+03
	Mean	**2.92E+03**	3.03E+03	3.00E+03	**2.92E+03**	2.99E+03
	Std	**4.00E+01**	9.22E+01	6.74E+01	4.17E+01	6.59E+01
CEC-10	Best	**2.90E+03**	2.91E+03	2.91E+03	**2.90E+03**	2.91E+03
	Mean	**2.97E+03**	**2.97E+03**	2.98E+03	**2.97E+03**	2.98E+03
	Std	**3.31E+01**	4.22E+01	5.29E+01	3.44E+01	5.87E+01

Table 9. Ablation experiment results (dim = 20).

We established grid maps of three different sizes: 10×10, 20×20, and 30×30, using MATLAB simulation software. Each map was populated with obstacles ranging from 10 to 30%. This resulted in nine distinct experimental environments, simulating both simple and complex obstacle scenarios. With a maximum iteration limit of 100, each method was subjected to 50 independent runs to reduce the impact of randomness.

Figure 17; Tables 10 and 11, and 12 display the comparisons and results of path planning data. As observed, the RCDBO method's average and ideal path lengths outperform the DBO algorithm in every map scenario. The iteration curves of the best and average paths designed in variously sized environments by the RCDBO and DBO algorithms are shown in Fig. 8 through 16. The beginning path length values predicted by RCDBO outperform those of the DBO method in every experimental situation. This shows that by increasing population variety and traversal capabilities, the RCDBO algorithm raises the caliber of early solutions.

When the map size is 10×10, as shown in Figs. 8, 9 and 10, in environments with simple obstacle distributions of 10% and 20%, the paths planned by the RCDBO method are relatively smooth, with even zero turning points. In the more challenging scenario with a 30% obstacle distribution, although the paths planned by RCDBO have more turning points, this is to achieve a shorter total path length. Analysis of the average iteration curves shows that, in settings with 10% and 20% obstacle distributions, the RCDBO algorithm converges faster at first than the DBO algorithm, suggesting that, after increasing population diversity, its iteration speed is optimized. Though RCDBO needs more iterations on average than DBO in a complicated environment with a 30% obstacle dispersion, it continuously discovers better solutions as the number of iterations rises. This suggests that whereas RCDBO disturbs the dung beetle population to prevent premature convergence and raises the possibility of finding globally optimal solutions, DBO is vulnerable to local optimal traps throughout the iteration phase.

The 20×20 and 30×30 maps pose more incredible difficulty in finding solutions compared to the 10×10 map. Figures 9, 10, 11, 12, 13 and 14 display the optimal pathways provided by the RCDBO and DBO algorithms for these two map sizes. The RCDBO algorithm increases the number of planned path turning points compared to the DBO algorithm to solve for the shortest paths in a 30×30-sized map environment with a complex obstacle ratio distribution of 20% and 30%. However, in other instances, due to relatively simple map environments, the

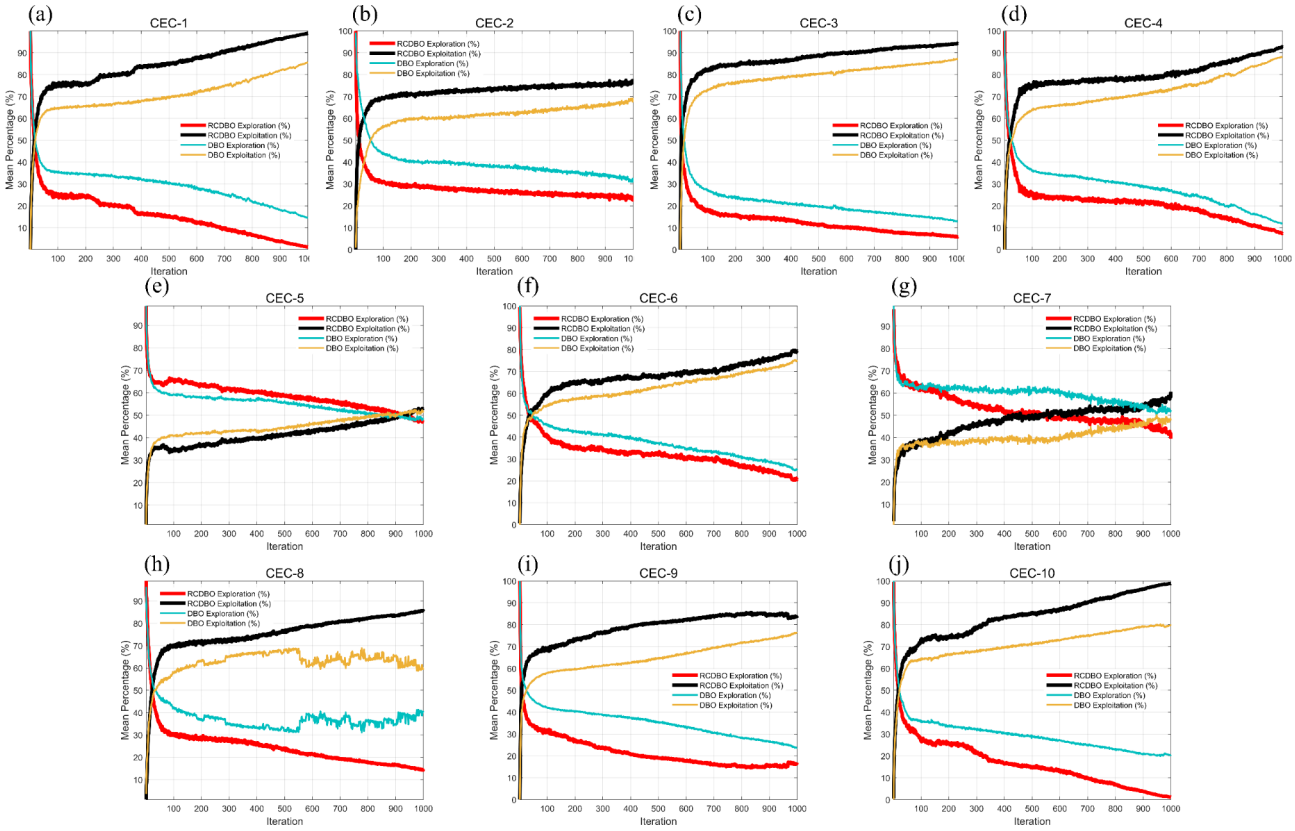


Fig. 7. Comparison of exploration and exploitation change curves between RCDBO and DBO.

Obstacle rate	Index	RCDBO	DBO	Decrease Percent
10%	Average Path length	13.923	14.497	3.96%
	Optimal path length	13.313	13.899	4.22%
	Number of Turns	4	5	20.00%
20%	Average Path length	12.728	14.182	10.25%
	Optimal path length	12.728	13.314	4.40%
	Number of Turns	0	3	100.00%
30%	Average Path length	14.509	15.364	5.56%
	Optimal path length	14.485	15.071	3.89%
	Number of Turns	7	6	-16.67%

Table 10. 10×10 map experiment results.

Obstacle rate	Index	RCDBO	DBO	Decrease Percent
10%	Average Path length	31.029	31.556	1.67%
	Optimal path length	30.971	31.556	1.85%
	Number of Turns	7	8	12.50%
20%	Average Path length	29.447	30.971	4.92%
	Optimal path length	28.627	30.971	7.57%
	Number of Turns	9	9	0.00%
30%	Average Path length	32.273	34.258	5.79%
	Optimal path length	29.799	33.799	11.83%
	Number of Turns	10	14	28.57%

Table 11. 20×20 map experiment results.

Obstacle rate	Index	RCDBO	DBO	Decrease Percent
10%	Average path length	44.621	46.284	3.59%
	Optimal path length	43.941	46.284	5.06%
	Number of turns	7	8	12.50%
20%	Average path length	46.780	48.042	2.63%
	Optimal path length	46.284	48.042	3.66%
	Number of turns	17	13	-30.77%
30%	Average path length	50.067	52.658	4.92%
	Optimal path length	49.213	52.627	6.49%
	Number of turns	23	12	-91.67%

Table 12. 30×30 map experiment results.

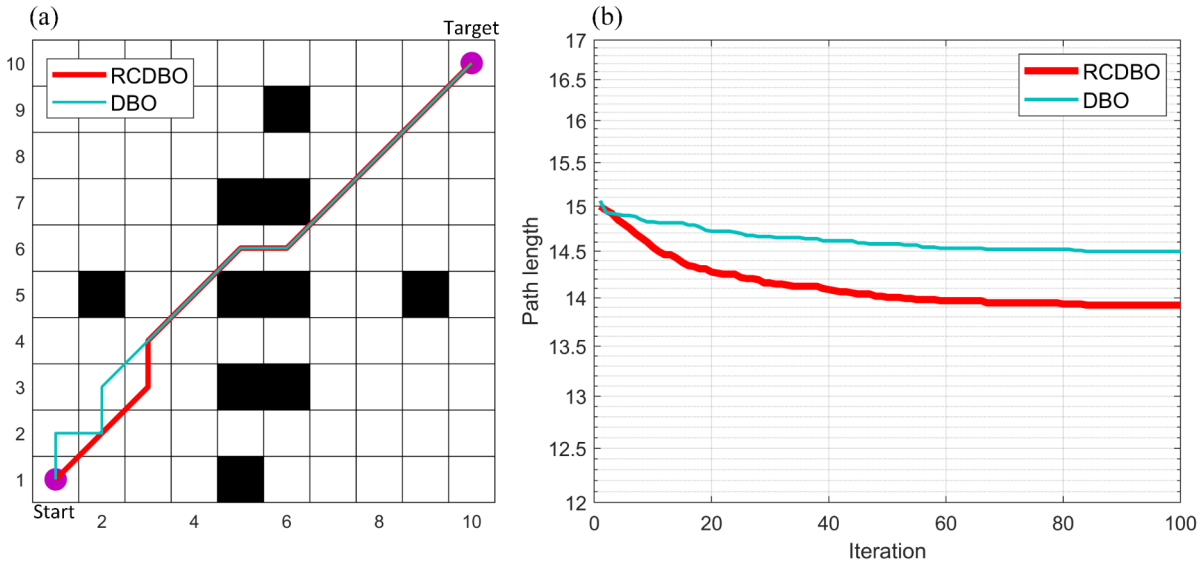


Fig. 8. Path planning in Scenario 1(Map size: 10 × 10, Obstacle rate: 10%). Panel **(a)** presents the optimal paths generated and Panel **(b)** presents the average iteration curves.

optimal paths generated by the RCDBO algorithm have fewer turning points. They are shorter and smoother than those generated by the DBO algorithm. Examining the average iteration curves, as map size and complexity escalate, so do the challenges. DBO tends to converge prematurely, falling into local optima, while RCDBO demonstrates a more pronounced advantage in escaping these traps. RCDBO consistently outperforms DBO with deeper iterations, indicating its strengthened global optimization capability through strategies like random walk and vertical and horizontal crossover. However, it can also be seen that in dense obstacle environments, when the shortest path is the only goal of path planning, the path inflection points planned by RCDBO increase, and the computational cost of this algorithm, which incorporates multiple strategies, increases. So, how to plan the shortest path while still making the path smooth and at the same time improving computational efficiency can be the next research direction.

Compared to DBO, RCDBO has shown significantly improved optimization performance, with higher path planning efficiency and superior results. Even in complex environments, RCDBO can avoid premature convergence and local minimum traps, demonstrating outstanding global path planning capabilities.

Conclusion

The basic dung beetle optimization algorithm, which is known to be prone to becoming trapped in local optima and to have weak global search skills, has an imbalance between its powers for local exploitation and global search. This is the problem that the proposed approach attempts to address. To overcome these limitations, a multi-strategy cooperative improvement RCDBO algorithm is introduced. The RCDBO method utilizes a Bernoulli-based chaotic mapping technique for population initialization. This approach enhances the uniform randomness of the initial population, hence boosting the quality and efficiency of the first solutions. Given the significant importance of dung beetles' ball-rolling behavior in global optimization, a random walk strategy is implemented to disrupt the rolling behavior of dung beetles when the current optimal solution remains unchanged for five consecutive iterations. This strategy aims to improve the effectiveness of early-stage optimization. Meanwhile,

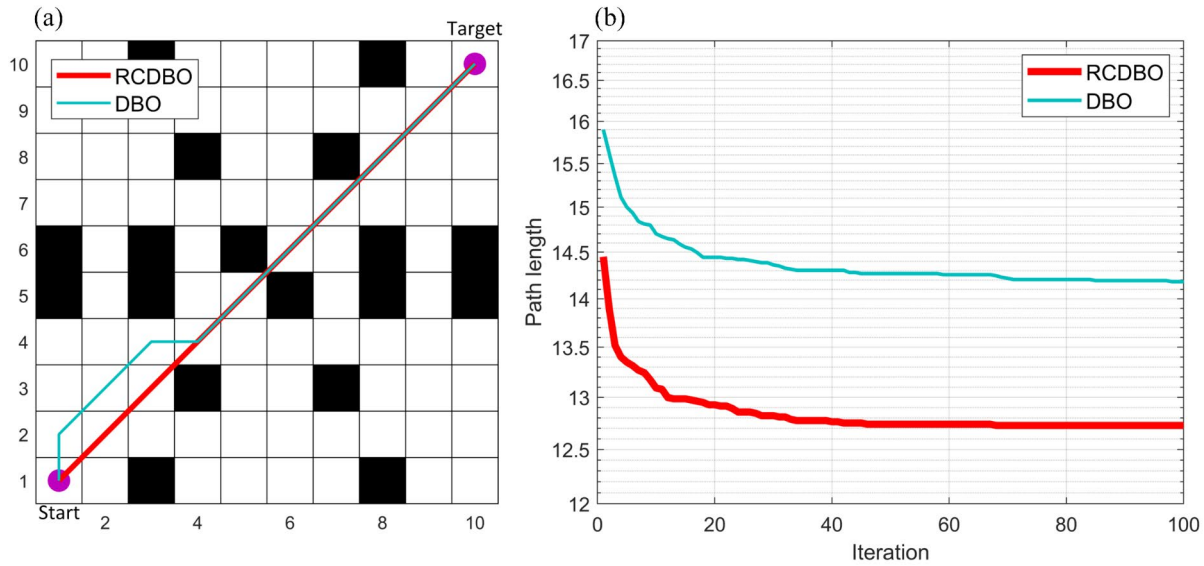


Fig. 9. Path planning in Scenario 2(Map size: 10×10, Obstacle rate: 20%)Panel **(a)** presents the optimal paths generated and Panel **(b)** presents the average iteration curves.

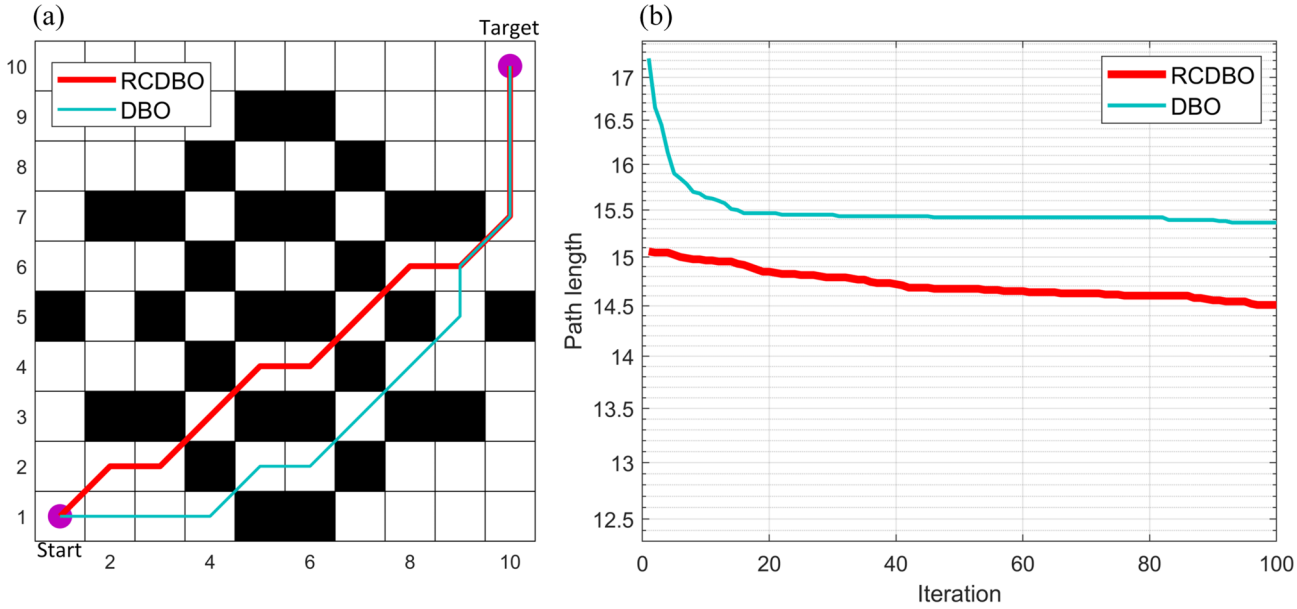


Fig. 10. Path planning in Scenario 3(Map size: 10×10, Obstacle rate: 30%)Panel **(a)** presents the optimal paths generated and Panel **(b)** presents the average iteration curves.

the vertical and horizontal crossover method was introduced, which has a risk of perturbing the current optimal position of the dung beetle population. This is done to prevent the algorithm from stagnating in the later stage. Implementing these three tactics is intended to prevent the algorithm from getting caught in local optima, improve its ability to optimize globally, and find a happy medium between exploring locally and globally.

To assess the effectiveness of the RCDBO algorithm, we chose to compare it with six other algorithms (IDBO, NCHHO, SABO, WOA, PO, DBO). The comparison used twelve benchmark functions as well as ten Bias Shift and Rotated Group functions from the CEC2021 test suite. Additionally, we compared and evaluated each RCDBO strategy's efficacy, as well as DBO and RCDBO's capacity to strike a balance between exploration and exploitation behaviors, using the CEC2021 test suite. Furthermore, Friedman's test and Wilcoxon's rank-sum test for statistical analysis were finished. The results indicate that the enhanced RCDBO algorithm exhibits more excellent problem-solving capabilities overall.

Moreover, when it comes to finding the best route in grid maps, the enhanced RCDBO algorithm surpasses the original DBO method in terms of the most efficient way and the average path across nine maps with varying

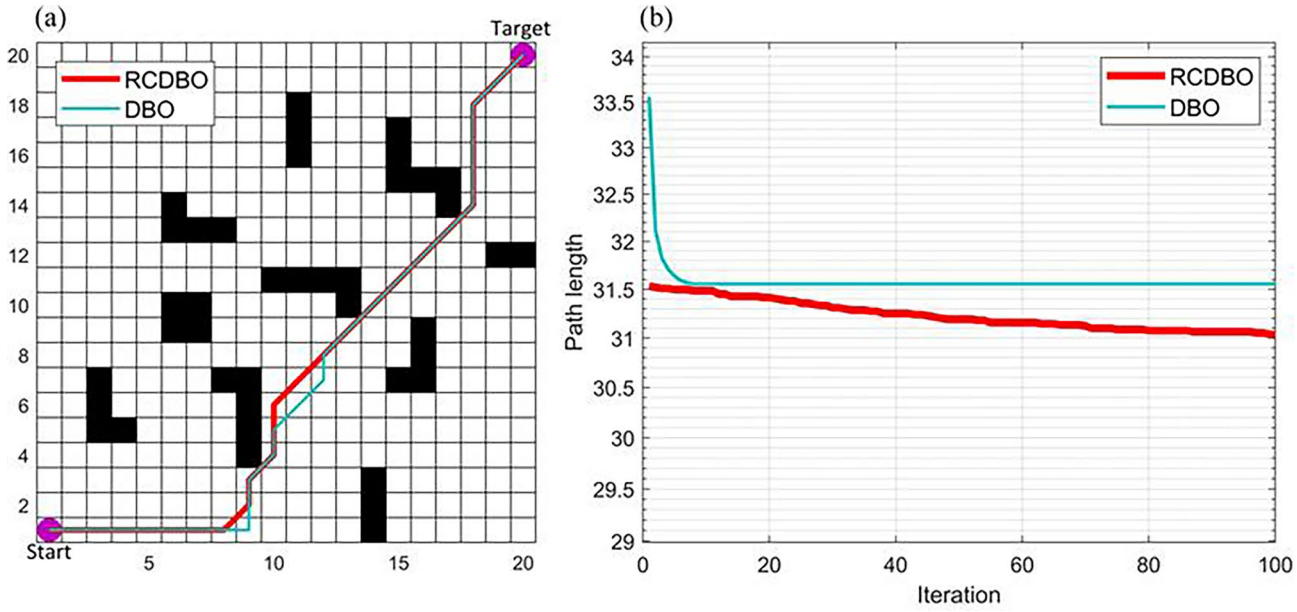


Fig. 11. Path planning comparison in Scenario 4(Map size: 20 × 20, Obstacle rate: 10%)Panel **(a)** presents the optimal paths generated and Panel **(b)** presents the average iteration curves.

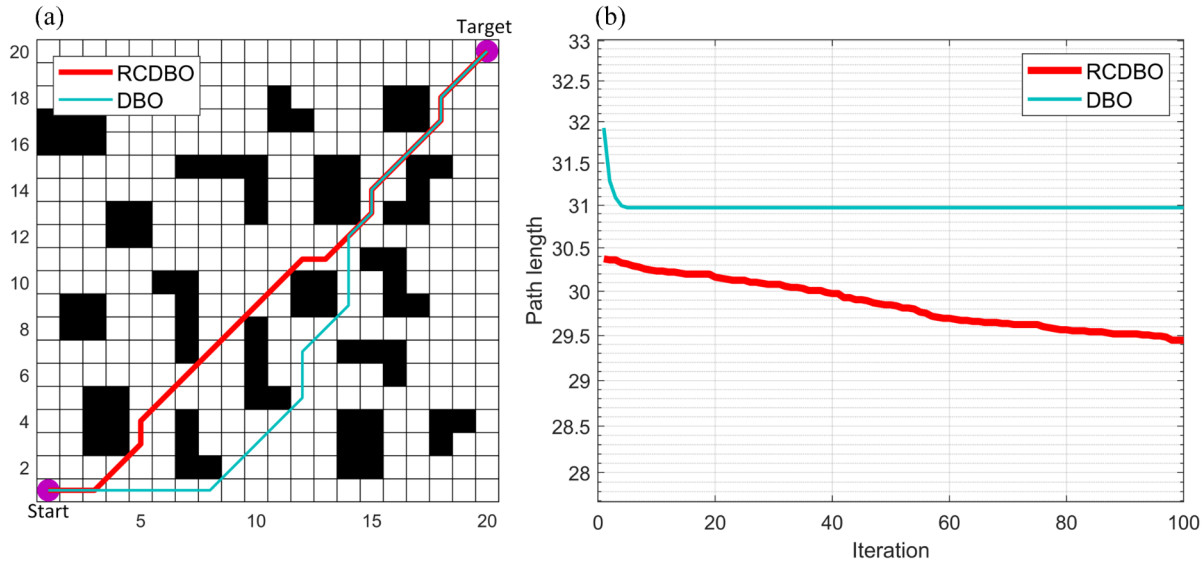


Fig. 12. Path planning comparison in Scenario 5(Map size: 20 × 20, Obstacle rate: 20%)Panel **(a)** presents the optimal paths generated and Panel **(b)** presents the average iteration curves.

complexity and sizes. The RCDBO method demonstrates superior iterative optimization speed and produces smoother pathways in simple obstacle map situations. The RCDBO algorithm can navigate through complicated obstacle map environments by avoiding premature convergence and identifying a more efficient global path than the original DBO method. The test results demonstrate that the RCDBO method surpasses the DBO method in performance when combined with the three approaches. It demonstrates exceptional performance in addressing problems in test functions and path-planning difficulties in navigation applications.

This research supports theoretical advancements in addressing autonomous navigation challenges in diverse fields such as mining, emergency response, and logistics and contributes to the progress of autonomous driving technology. Considering the ever-complex and dynamic real-world environments, future studies can explore multi-objective dynamic obstacle avoidance problems for further advancements.

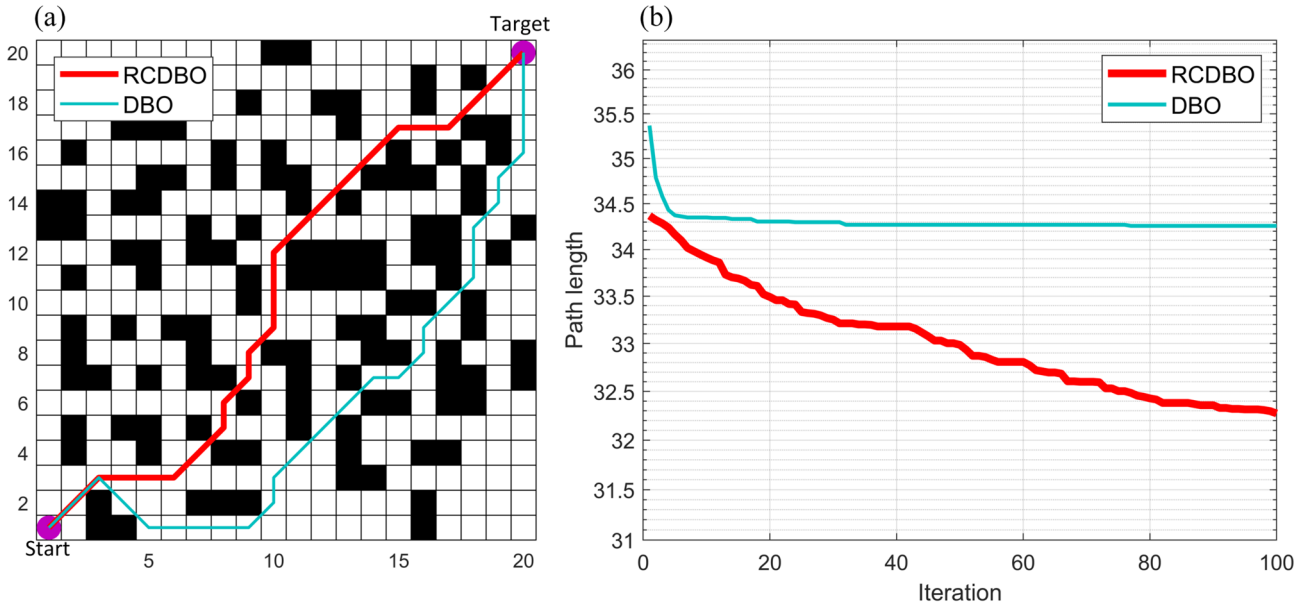


Fig. 13. Path planning comparison in Scenario 6(Map size: 20×20, Obstacle rate: 30%) Panel **(a)** presents the optimal paths generated and Panel **(b)** presents the average iteration curves.

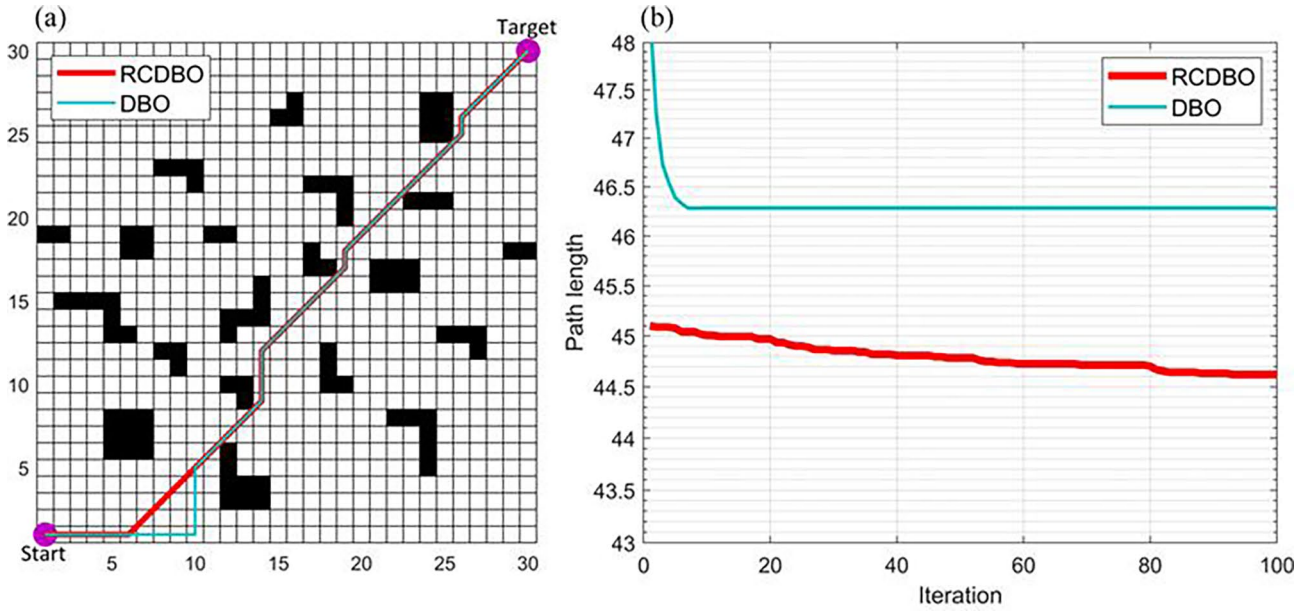


Fig. 14. Path planning comparison graph in Scenario 7(Map size: 30×30, Obstacle rate: 10%)Panel **(a)** presents the optimal paths generated and Panel **(b)** presents the average iteration curves.

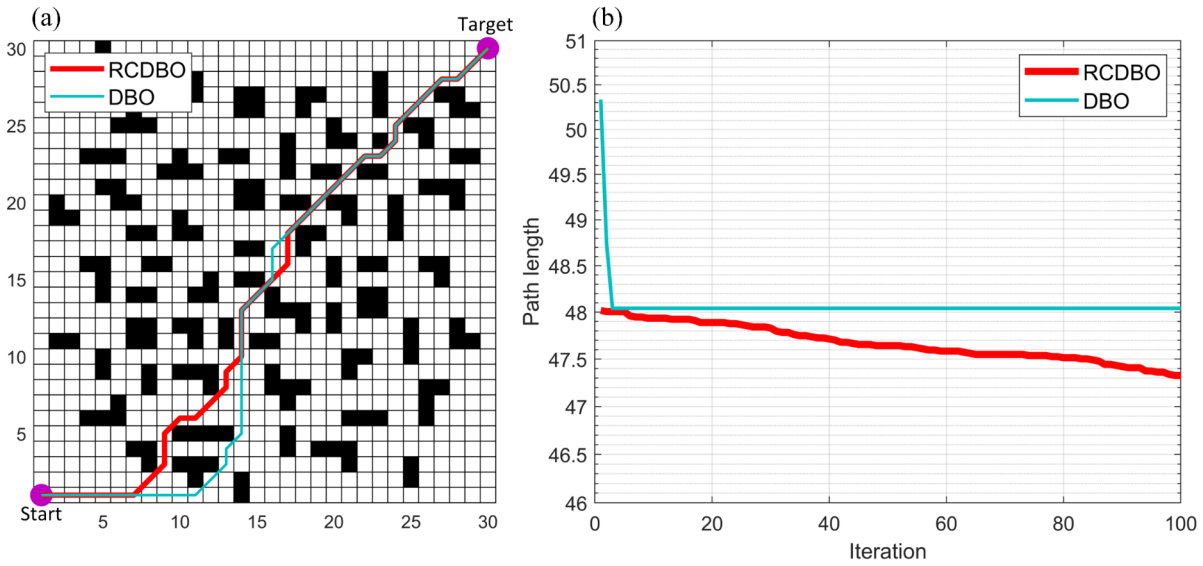


Fig. 15. Path planning comparison graph in Scenario 8(Map size: 30×30, Obstacle rate: 20%) Panel **(a)** presents the optimal paths generated and Panel **(b)** presents the average iteration curves.

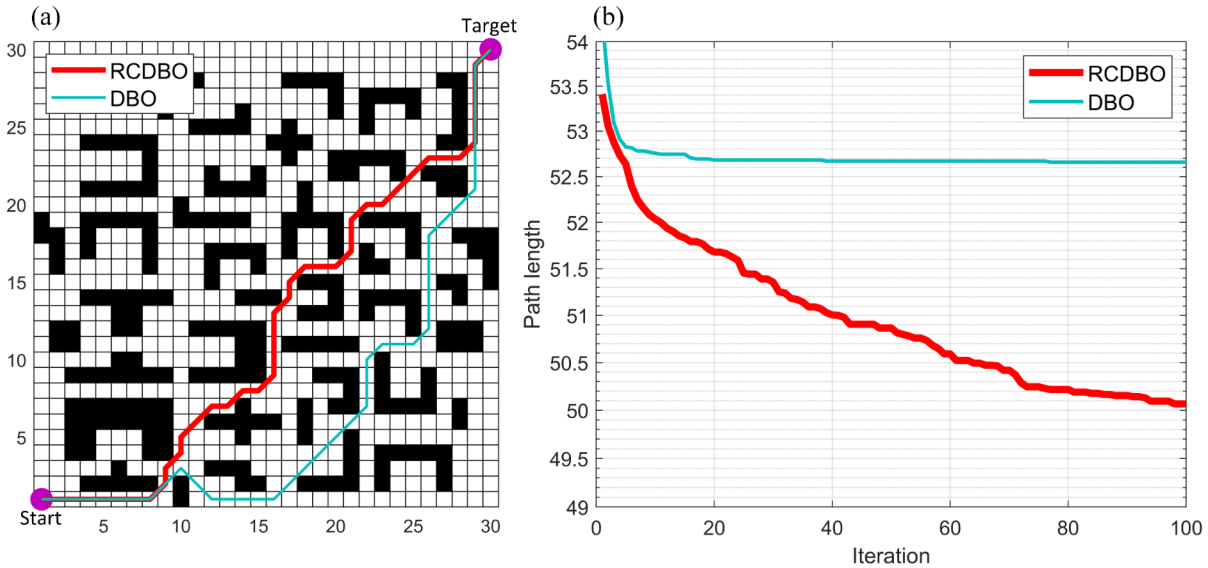


Fig. 16. Path planning comparison graph in Scenario 9 (Map size: 30×30, Obstacle rate: 30%). Panel **(a)** presents the optimal paths generated and Panel **(b)** presents the average iteration curves.

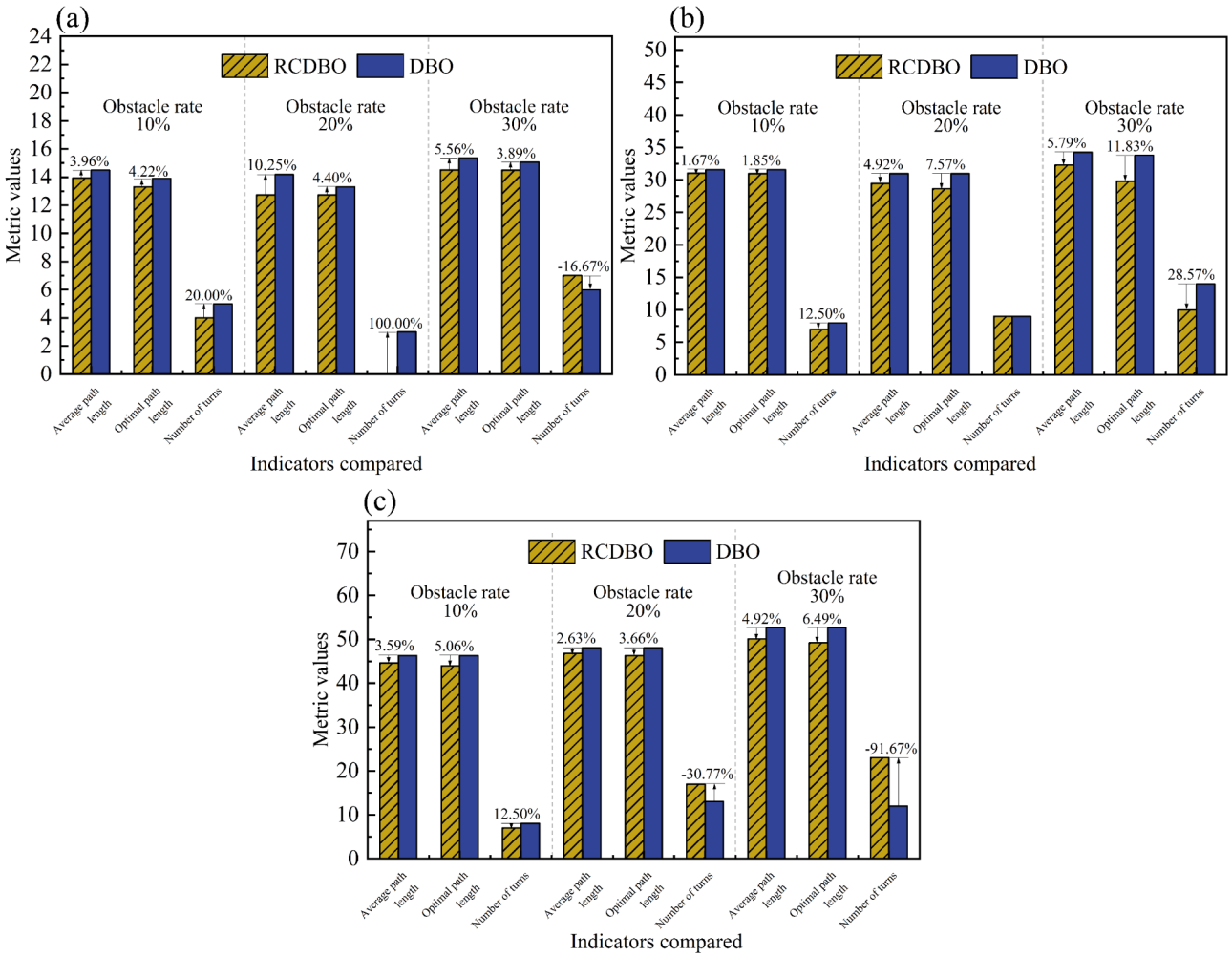


Fig. 17. The histogram of path planning results. Panel **(a)** to **(c)** present the results in the 10×10 map, 20×20 map and 30×30 map, respectively.

Data availability

The dataset and code used and/or analyzed in this study are accessible at https://zenodo.org/records/14048789.

Received: 17 May 2024; Accepted: 8 November 2024
Published online: 14 November 2024

References

1. Reda, M., Onsy, A., Haikal, A. Y. & Ghanbari, A. Path planning algorithms in the autonomous driving system: a comprehensive review. *Robot. Auton. Syst.* **174**, 104630. https://doi.org/10.1016/j.robot.2024.104630 (2024).
2. Ma, H., Pei, W. & Zhang, Q. Research on path planning algorithm for driverless vehicles. *Mathematics*. **10**(15), 2555. https://doi.org/10.3390/math10152555 (2022).
3. Aggarwal, S. & Kumar, N. Path planning techniques for unmanned aerial vehicles: a review, solutions, and challenges. *Comput. Commun.* **149**, 270–299. https://doi.org/10.1016/j.comcom.2019.10.014 (2020).
4. Qin, H. et al. Review of autonomous path planning algorithms for mobile robots. *Drones*. **7**(3), 211. https://doi.org/10.3390/drones7030211 (2023).
5. He, Z., Liu, C., Chu, X. & Negenborn, R. Dynamic anti-collision A-star algorithm for multi-ship encounter situations. *Appl. Ocean Res.* **118**, 102995. https://doi.org/10.1016/j.apor.2021.102995 (2022).
6. Sundarraj, S. et al. Route Planning for an Autonomous Robotic Vehicle employing a weight-controlled particle swarm-optimized Dijkstra Algorithm. *IEEE Access.* **1**, 92433–92442. https://doi.org/10.1109/ACCESS.2023.3302698 (2023).
7. Zhang, R. et al. Intelligent path planning by an improved RRT algorithm with dual grid map. *Alexandria Eng. J.* **88**, 91–104. https://doi.org/10.1016/j.aej.2023.12.044 (2024).
8. Zhang, W., Xu, G., Song, Y. & Wang, Y. An obstacle avoidance strategy for complex obstacles based on artificial potential field method. *J. Field Robot.* **40**(5), 1231–1244. https://doi.org/10.1002/rob.22183 (2023).
9. Chu, Z., Wang, F., Lei, T. & Luo, C. Path planning based on deep reinforcement learning for autonomous underwater vehicles under ocean current disturbance. *IEEE Trans. Intell. Veh.* **8**(1), 108–120. https://doi.org/10.1109/TIV.2022.3153352 (2022).
10. Cui, Z. & Gao, X. Theory and applications of s-warm intelligence. *Neural Comput. Appl.* **21**(2), 58–63. https://doi.org/10.1007/s00521-011-0523-8 (2012).

11. Liu, L. et al. Path planning techniques for mobile robots: review and prospect. *Expert Syst. Appl.* **227**, 120254. https://doi.org/10.1016/j.eswa.2023.120254 (2023).
12. Xu, L., Cao, M. & Song, B. A new approach to smooth path planning of mobile robot based on quartic Bezier transition curve and improved PSO algorithm. *Neurocomputing*. **473**, 98–106. https://doi.org/10.1016/j.neucom.2021.12.016 (2022).
13. Wu, L., Huang, X., Cui, J., Liu, C. & Xiao, W. Modified adaptive ant colony optimization algorithm and its application for solving path planning of mobile robot. *Expert Syst. Appl.* **215**, 119410. https://doi.org/10.1016/j.eswa.2022.119410 (2023).
14. Cui, Y., Hu, W. & Rahmani, A. Multi-robot path planning using learning-based artificial bee colony algorithm. *Eng. Appl. Artif. Intell.* **129**, 107579. https://doi.org/10.1016/j.engappai.2023.107579 (2024).
15. Li, G., Liu, C., Wu, L. & Xiao, W. A mixing algorithm of ACO and ABC for solving path planning of mobile robot. *Appl. Soft Comput.* **148**, 110868 (2023).
16. Zamani, H., Nadimi-Shahraki, M. H., Gandomi, A. H. & QANA Quantum-based avian navigation optimizer algorithm. *Eng. Appl. Artif. Intell.* **104**, 104314. https://doi.org/10.1016/j.engappai.2021.104314 (2021).
17. Yu, X., Jiang, N., Wang, X. & Li, M. A hybrid algorithm based on grey wolf optimizer and differential evolution for UAV path planning. *Expert Syst. Appl.* **215**, 119327 (2023).
18. Mirjalili, S., Mirjalili, S. M. & Lewis, A. Grey wolf optimizer. *Adv. Eng. Softw.* **69**, 46–61. https://doi.org/10.1016/j.advengsoft.2013.12.007 (2014).
19. Arora, S. & Anand, P. Binary butterfly optimization approaches for feature selection. *Expert Syst. Appl.* **116**, 147–160. https://doi.org/10.1016/j.eswa.2018.08.051 (2019).
20. Xue, J. & Shen, B. A novel swarm intelligence optimization approach: sparrow search algorithm. *Syst. Sci. Control Eng.* **8**(1), 22–34. https://doi.org/10.1080/21642583.2019.1708830 (2020).
21. Mirjalili, S. et al. Salp swarm algorithm: a bio-inspired optimizer for engineering design problems. *Adv. Eng. Softw.* **114**, 163–191. https://doi.org/10.1016/j.advengsoft.2017.07.002 (2017).
22. Luo, Y., Qin, Q., Hu, Z. & Zhang, Y. Path planning for Unmanned Delivery Robots based on EWB-GWO algorithm. *Sensors*. **23**(4), 1867. https://doi.org/10.3390/s23041867 (2023).
23. Liu, L. et al. Dynamic path planning of mobile robot based on improved sparrow search algorithm. *Biomimetics*. **8**(2), 182. https://doi.org/10.3390/biomimetics8020182 (2023).
24. Duan, X. H., Wu, J. X. & Xiong, Y. L. Dynamic emergency vehicle path planning and traffic evacuation based on salp swarm algorithm. *J. Adv. Transp.* **28**, 7862746. https://doi.org/10.1155/2022/786274 (2022).
25. Xue, J. & Shen, B. Dung beetle optimizer: a new meta-heuristic algorithm for global optimization. *J. Supercomputing*. **79**(7), 7305–7336. https://doi.org/10.1007/s11227-022-04959-6 (2023).
26. Duan, J., Gong, Y., Luo, J. & Zhao, Z. Air-quality prediction based on the ARIMA-CNN-LSTM combination model optimized by dung beetle optimizer. *Sci. Rep.* **13**(1), 12127. https://doi.org/10.1038/s41598-023-36620-4 (2023).
27. Shen, Q., Zhang, D., Xie, M. & He, Q. Multi-strategy enhanced Dung Beetle Optimizer and its application in three-dimensional UAV path planning. *Symmetry*. **15**(7), 1432. https://doi.org/10.3390/sym15071432 (2023).
28. Zamani, H., Nadimi-Shahraki, M. H. & Gandomi, A. H. CCSA: conscious neighborhood-based crow search algorithm for solving global optimization problems. *Appl. Soft Comput.* **85**, 105583. https://doi.org/10.1016/j.asoc.2019.105583 (2019).
29. Sun, H., He, D., Ma, H., Wen, Z. & Deng, J. The parameter identification of Metro rail corrugation based on effective signal extraction and inertial reference method. *Eng. Fail. Anal.* **158**, 108043. https://doi.org/10.1016/j.engfailanal.2024.108043 (2024).
30. Zhu, F. et al. Dung beetle optimization algorithm based on quantum computing and multi-strategy fusion for solving engineering problems. *Expert Syst. Appl.* **236**, 121219. https://doi.org/10.1016/j.eswa.2023.121219 (2024).
31. Li, L. et al. Enhancing Swarm Intelligence for Obstacle Avoidance with Multi-strategy and Improved Dung Beetle optimization Algorithm in Mobile Robot Navigation. *Electronics*. **12**(21), 4462. https://doi.org/10.3390/electronics12214462 (2023).
32. Shen, Q. et al. A novel multi-objective dung beetle optimizer for Multi-UAV cooperative path planning. *Heliyon*. **10**(17), 37286. https://doi.org/10.1016/j.heliyon.2024.e37286 (2024).
33. Lyu, L., Jiang, H. & Yang, F. Improved Dung Beetle Optimizer Algorithm with Multi-strategy for global optimization and UAV 3D path planning. *IEEE Access.* **12**, 69240–69257. https://doi.org/10.1109/ACCESS.2024.3401129 (2024).
34. Li, Y., Han, M. & Guo, Q. Modified whale optimization algorithm based on tent chaotic mapping and its application in structural optimization. *KSCE J. Civ. Eng.* **24**(12), 3703–3713. https://doi.org/10.1007/s12205-020-0504-5 (2020).
35. Wang, P., Zhang, Y. & Yang, H. Research on economic optimization of microgrid cluster based on chaos sparrow search algorithm. *Computational Intelligence and Neuroscience*, **2021**, 5556780. https://doi.org/10.1155/2021/5556780 (2021).
36. Pan, J., Li, S., Zhou, P., Yang, G. & Lu, D. Dung Beetle optimization Algorithm guided by improved sine algorithm. *Comput. Eng. Appl.* **59**(22), 92–110 (2023).
37. Ma, X. J., He, H. & Wang, H. W. Maximum exponential entropy segmentation method and based on improved sparrow search algorithm. *Sci. Technol. Eng.* **23**(16), 6983–6992 (2023).
38. Meng, A. B., Chen, Y. C., Yin, H. & Chen, S. Z. Crisscross optimization algorithm and its application. *Knowl. Based Syst.* **67**, 218–229. https://doi.org/10.1016/j.knosys.2014.05.004 (2014).
39. Zhao, D. et al. Ant colony optimization with horizontal and vertical crossover search: fundamental visions for multi-threshold image segmentation. *Expert Syst. Appl.* **167**, https://doi.org/10.1016/j.eswa.2020.114122 (2021).
40. Liang, X., Zhang, Y. & Lone, W. Spider monkey optimization algorithm with crisscross optimization. *Math. Practice Theory*. **52**(12), 144–158 (2022).
41. Hu, G., Zhong, J., Du, B. & Wei, G. An enhanced hybrid arithmetic optimization algorithm for engineering applications. *Comput. Methods Appl. Mech. Eng.* **394**, 114901. https://doi.org/10.1016/j.cma.2022.114901 (2022).
42. Mittal, N., Singh, U. & Sohi, B. S. Modified grey wolf optimizer for global engineering optimization. *Appl. Comput. Intell. Soft Comput.* **1**, 7950348. https://doi.org/10.1155/2016/7950348 (2016).
43. Long, W. & Wu, T. Improved grey wolf optimization algorithm coordinating the ability of exploration and exploitation. *Control Decis.* **32**(10), 1749–1757. https://doi.org/10.13195/j.kzyjc.2016.1545 (2017).
44. Mirjalili, S. & Lewis, A. The whale optimization algorithm. *Adv. Eng. Softw.* **95**, 51–67. https://doi.org/10.1016/j.advengsoft.2016.01.008 (2016).
45. Trojovský, P., Dehghani, M., Subtraction-Average-Based, M. & Optimizer: a new swarm-inspired metaheuristic algorithm for solving optimization problems. *Biomimetics*. **8**(2), 149. https://doi.org/10.3390/biomimetics8020149 (2023).
46. Lian, J. et al. Parrot optimizer: Algorithm and applications to medical problems. *Comput. Biol. Med.* **172**, 108064 (2024).
47. Dehkordi, A. A. et al. Nonlinear-based chaotic harris hawks optimizer: algorithm and internet of vehicles application. *Appl. Soft Comput.* **109**, 107574. https://doi.org/10.1016/j.compbiomed.2024.108064 (2021).
48. Kaur, A. & Kumar, Y. A new metaheuristic algorithm based on water wave optimization for data clustering. *Evol. Intel.* **15**(1), 759–783. https://doi.org/10.1007/s12065-020-00562-x (2022).
49. Zamani, H. & Nadimi-Shahraki, M. H. Starling murmuration optimizer: a novel bio-inspired algorithm for global and engineering optimization. *Comput. Methods Appl. Mech. Eng.* **392**, 114616. https://doi.org/10.1016/j.cma.2022.114616 (2022).
50. Morales-Castañeda, B., Zaldivar, D., Cuevas, E., Fausto, F. & Rodríguez, A. A better balance in metaheuristic algorithms: does it exist? *Swarm Evol. Comput.* **54**, 100671. https://doi.org/10.1016/j.swevo.2020.100671 (2020).
51. Ou, Y., Fan, Y., Zhang, X., Lin, Y. & Yang, W. Improved A* path planning method based on the grid map. *Sensors*. **22**(16), 6198. https://doi.org/10.3390/s22166198 (2022).

Acknowledgements
This work was supported by the Science and Technology Programme Projects in Meishan city (2023KJZD100).

Author contributions
X.T. wrote the main manuscript text and programme; Z.H. wrote the programme; C.J: prepared Figs. 6 and 17; All authors reviewed the manuscript.

Declarations

Competing interests
The authors declare no competing interests.

Additional information
Supplementary Information The online version contains supplementary material available at https://doi.org/10.1038/s41598-024-79420-0.

Correspondence and requests for materials should be addressed to X.T.

Reprints and permissions information is available at www.nature.com/reprints.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by-nc-nd/4.0/.

© The Author(s) 2024